A Sound Semantics Approximation of Solidity for Enhanced Vulnerability Detection

MAITRI KUSHWAHA, Indian Institute of Technology Patna, India
RAJU HALDER✉, Indian Institute of Technology Patna, India
AGOSTINO CORTESI, Ca' Foscari University of Venice, Italy

As Solidity smart contracts become central to decentralized finance and governance, the exploitation of critical vulnerabilities has repeatedly resulted in severe financial losses. Existing state-of-the-art methods that rely on symbolic execution or syntactic patterns are unable to model the language semantics accurately. This limits their ability to reason about all possible runtime behaviors and, hence, often produces numerous false alarms under adversarial blockchain environment. To address these challenges, in this paper, we propose a semantics-driven static analysis framework for detecting vulnerabilities in Solidity. Our proposed framework is grounded in Abstract Interpretation theory, enabling safe approximation of program semantics to formally capture dynamic behavioral properties of Solidity smart contracts at different levels of abstraction. While it performs fixpoint semantics computations on an augmented control flow graph guided by adversarial contexts, it applies semantic dependences refinement through numerical abstractions as well. This facilitates more precise reasoning regarding semantic properties indicating the presence of possible Solidity vulnerabilities. Our prototype, developed using the APRON library, supports various numerical abstract domains, such as Intervals, Octagons, and Polyhedra, thereby optimizing the trade-off between precision and computational cost. Empirical results on four benchmark datasets (SBC, Qian, RSD, and SolidiFI) show that the Interval domain works well for integer overflow and underflow and for reentrancy, while the Polyhedra domain captures the linear relations among variables needed for timestamp and transaction-ordering dependencies. Furthermore, the semantics-aware analysis enables our tool to achieve the highest F1-score in seven of the nine evaluated dataset-vulnerability combinations, with average F1-score improvements over the strongest existing baselines of about 5.3, 15.0, 13.6, and 5.5 percentage points for Reentrancy, Integer Overflow and Underflow, Timestamp Dependency, and Transaction Ordering Dependency, respectively, together with AUC improvements of up to 0.41.

CCS Concepts: • **Security and privacy → Software security engineering**; • **Theory of computation → Program analysis**; **Denotational semantics**; **Abstraction**.

Additional Key Words and Phrases: Blockchain, Smart Contract, Solidity, Vulnerability Detection, Semantics, Static Analysis, Abstract Interpretation

1 Introduction

Solidity, the most popular smart contract language, has become the primary choice for developers to build decentralized applications on Ethereum. These contracts enable automation across finance, governance, and supply-chain management, eliminating the need for trusted intermediaries. However, their immutable nature makes them susceptible to various exploitation. Even minor bugs can lead to significant financial losses, as demonstrated by DAO hack [14], Parity wallet vulnerability [43], Beautychain's token BEC [4], NFT gaming blockchain breach [15], and many more.

Authors' Contact Information: Maitri Kushwaha, Indian Institute of Technology Patna, India, maitri_2321cs03@iitp.ac.in; Raju Halder, Indian Institute of Technology Patna, India, halder@iitp.ac.in; Agostino Cortesi, Ca' Foscari University of Venice, Italy, cortesi@unive.it.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2769-6480/2026/9-ART
https://doi.org/10.1145/3849490

Just Accepted

To address these vulnerabilities, several static analysis and formal verification tools and techniques have been proposed in the literature, including Oyente [29], Mythril [33], Osiris [46], Slither [18], and Zeus [25]. However, studies and surveys [3, 7, 16, 19] show that these tools often miss many real-world vulnerabilities. An evaluation of 47,000 contracts [16] found that even the best tools detect fewer than 27% of actual issues in large-scale assessments. This performance gap is rooted in fundamental limitations: symbolic execution based analyzers, such as Oyente and Mythril, suffer from path explosion, limited modeling of persistent states and loops, and the overhead of constraint solving, while rule-based tools like Slither rely on manually encoded patterns that often fail to generalize across semantically equivalent constructs. Zeus, which typically operates over simplified contract models based on policy assertions verification via LLVM Bitcode verifier, lacks fine-grained semantic reasoning leading to missed vulnerabilities or excessive false positives.

Critically, most existing approaches do not model the language semantics adequately, limiting their ability to reason about all possible runtime behaviors under various adversarial blockchain conditions. This limitation in syntactic and symbolic methods has driven a paradigm shift toward semantics-based analysis across the broader blockchain ecosystem, extending well beyond Solidity. Recent works successfully apply abstract interpretation to smart contracts written in general-purpose languages, addressing determinism in Cosmos SDK [37], read-write conflicts in Hyperledger Fabric [35], and untrusted cross-contract invocations [36]. While these approaches target the specific semantics and memory models of Go, they share a fundamental philosophy with our work: relying on a sound approximation of program semantics rather than pattern matching or under-approximated symbolic execution. Motivated by this, in this paper, we propose a semantics-aware smart contract vulnerability detection (SCVD) framework for Solidity by extending the Abstract Interpretation theory [11], a unifying framework for computing safe approximations of dynamic behavioral properties of Solidity smart contracts at different levels of abstraction. This is achieved by substituting the concrete domains of computations and their basic concrete semantic operations with appropriate abstract domains and corresponding abstract semantics operations. The basic intuition is that abstract domains are representations of some properties of interest about concrete domains' values, while abstract operations simulate, over the properties encoded by abstract domains, the behavior of their concrete counterparts. The use of Abstract Interpretation framework provides a systematic approach offering its tuning at different levels of abstractions so as to achieve a trade-off between precision and computation cost. Our proposed static analysis framework, enriched by augmented control- and data-flow analysis over appropriate semantic domains, demonstrates sound vulnerability detection in smart contracts prior to their deployment. Figure 1 presents a motivating example that demonstrates the critical need for semantic analysis. As shown in Part A, starting from the initial value `fee = 3`, the loop repeatedly multiplies `fee` by 3. For any input `periods` ≥ 5, the resulting computation exceeds the maximum value representable by `uint8` (255), leading to an integer overflow vulnerability. As detailed in Part B, latest versions of Mythril and Oyente+ analyze compiled EVM bytecode and do not report the corresponding source-level overflow condition in this example. While Osiris cannot parse Solidity 0.8 syntax, it also fails to detect the vulnerability even after downgrading it to a lower version. Slither's pattern-based AST analysis does not provide the loop-sensitive arithmetic range tracking required to identify this overflow. Even though the Solidity 0.8 compiler has already inserted runtime guards, static overflow detection in the source code remains relevant for three reasons. First, under Solidity 0.8, a runtime revert triggered by an arithmetic overflow still incurs gas costs for all computations performed before the failure point, imposing an unnecessary economic burden on users. Second, although the runtime check prevents silent state corruption, the contract logic still contains an execution path that is guaranteed to fail for a given subset of inputs. Third, Solidity explicitly permits runtime arithmetic checks to be disabled through `unchecked` blocks. Within such regions, arithmetic follows wraparound semantics similar to those of pre-0.8 Solidity, without automatic revert protection. This feature is not merely theoretical; gas-optimized production contracts frequently use unchecked arithmetic to reduce execution costs when developers consider the relevant operations safe. Widely adopted libraries, including OpenZeppelin, employ `unchecked` blocks in performance-critical code paths for this purpose.

Just Accepted

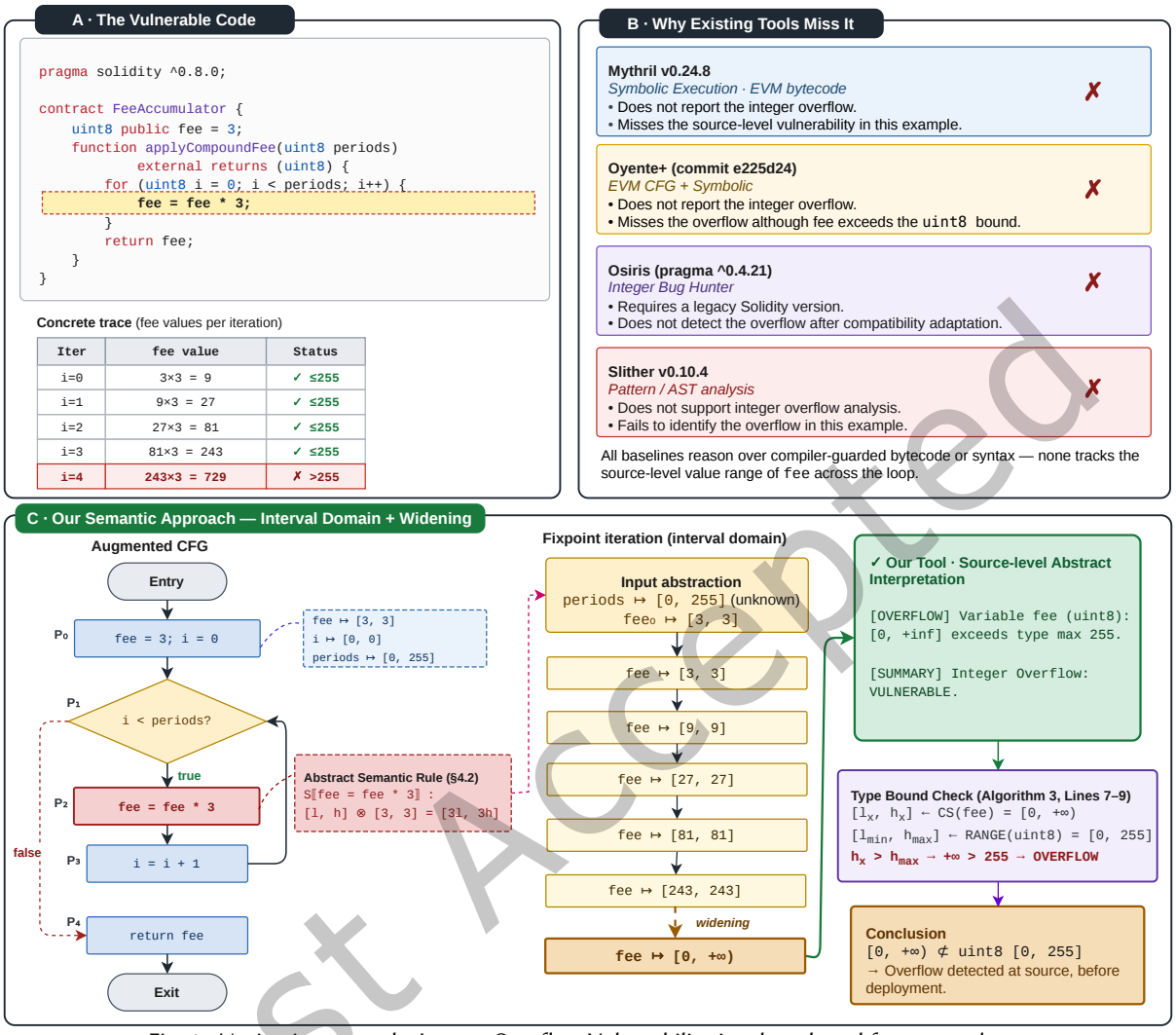


Fig. 1. Motivating example: Integer Overflow Vulnerability in a loop-based fee accumulator.

Just Accepted

Therefore, static analysis remains an essential safeguard in modern smart-contract development for both software assurance and security auditing. In this direction, our approach, shown in Part C, constructs an augmented control flow graph (CFG) from the Solidity source code and performs fixpoint-based abstract interpretation over the interval domain. By tracking the range of `fee` across loop iterations, the analysis infers that its possible value exceeds the `uint8` type bound, independently of compiler-inserted runtime protection, and reports the overflow condition before deployment.

In essence, our work is motivated by the following fundamental research questions:

(1) **RQ1: Detection Accuracy.** Can a semantics-based, multi-domain abstract interpretation framework achieve higher precision and recall than existing tools across diverse smart contract vulnerabilities?

(2) **RQ2: Domain Trade-offs.** How does the choice of abstract domain (interval, octagon, polyhedra) affect the trade-off between analysis precision and computational cost?
(3) **RQ3: Generalizability.** Does our approach generalize across different datasets and vulnerability types, including those challenging for current tools?

To summarize, we have made the following contributions:

(1) We define formal semantics of Solidity in concrete domain, followed by its safe approximation in both non-relational (domain of *Intervals*) and relational (domains of *Octagons* and *Polyhedra*) abstract domains using Abstract Interpretation theory, that captures both blockchain-influenced and local program state.
(2) We propose a semantics-based static analysis framework that performs fixpoint computation over an augmented control flow graph (CFG) incorporating both data and control dependencies, enabling precise reasoning about contract behaviour under adversarial conditions.
(3) We adopt a semantics-based refinement of both data and control dependencies at intra- and inter-level over numerical abstract domains, which enhances SCVD precision by identifying a number of false dependencies in the cases of Transaction Ordering Dependency (TOD) and timestamp dependence vulnerability.
(4) We develop a modular analysis tool using Apron library [24] that supports multiple numerical abstract domains, including interval, octagon and the more expressive polyhedra domain, allowing trade-offs between performance and precision.
(5) We conduct our experiment on four publicly available benchmark datasets, namely SmartBugs Curated [17], Qian et al. [39], the Reentrancy Benchmark [40], and SolidiFI [20], and demonstrate improved precision and recall across four critical vulnerability classes: reentrancy, integer overflow and underflow, timestamp dependency, and TOD.

This paper is a revised and extended version of our preliminary work published in [27]. We significantly expand upon the conference version by:

- Extending the analysis from the non-relational *Intervals* domain to relational domains such as *Octagons* and *Polyhedra*, thereby enabling a more expressive and precise representation of contract state and inter-variable relationships. These relational abstractions allow the framework to capture both blockchain-influenced and local program semantics with better accuracy.
- We extend the dependency analysis by integrating a flow- and context-sensitive data and control dependencies with semantic refinement through abstract interpretation. This strengthens the framework's ability to distinguish genuine from spurious dependencies, particularly for timestamp dependency and Transaction Ordering Dependency (TOD), and improves reasoning about vulnerabilities arising from adversarial transaction ordering.
- We significantly expand the experimental evaluation by extending the SmartBugs Curated [17] benchmark suite and incorporating the large-scale dataset from Qian et al. [39] across three vulnerability classes, together with the Reentrancy Benchmark Dataset [40] of modern Solidity 0.8 reentrancy variants and the SolidiFI [20] bug-injection benchmark for transaction-ordering dependency. We also broaden the comparison from four to eight state-of-the-art baselines. This extensive validation is essential to demonstrate the framework's robustness and generalizability beyond the preliminary results of the conference version.

The rest of the paper is organized as follows. Section 2 introduces the motivation and background on the role of semantics in smart contract vulnerability detection. Sections 3 and 4 present the formal and abstract semantics of Solidity. Section 5 describes the proposed vulnerability detection framework covering reentrancy, integer overflow and underflow, timestamp, and transaction-ordering dependencies. Section 6 outlines the implementation details of our prototype tool. Section 7 reports the experimental evaluation, and Section 8 reviews the related work. Finally, Section 9 concludes the paper.

Just Accepted

```solidity
pragma solidity ^0.8.20;

contract Reentrancy_safe {

mapping(address => uint) private rewardsForA;

function depositReward(address _beneficiary)
public payable {
require(msg.value > 0, "send ETH to deposit
rewards");
rewardsForA[_beneficiary] += msg.value;
}

function withdrawReward() public {
uint amountToWithdraw=rewardsForA[msg.sender];
require(amountToWithdraw > 0);
rewardsForA[msg.sender] = 0;
(bool success,)=msg.sender.call{value:
amountToWithdraw}("");
require(success);
        }
}

```

(a) Reentrancy false-positive case.

```solidity
contract Governmental {
  address public owner;
  address public lastInvestor;
  uint public jackpot = 1 ether;
  uint public lastInvestmentTimestamp;
  uint public ONE_MINUTE = 1 minutes;

  function Governmental() {
    owner = msg.sender;
    if (msg.value<1 ether) throw;}

  function invest() {
    if (msg.value<jackpot/2) throw;
    lastInvestor = msg.sender;
    jackpot += msg.value/2;
    // <yes> <report> TIME_MANIPULATION
    lastInvestmentTimestamp = block.timestamp;}

  function resetInvestment() {
    if(block.timestamp<lastInvestmentTimestamp
    + ONE_MINUTE)
      throw;
    lastInvestor.send(jackpot);
    owner.send(this.balance-1 ether);
    lastInvestor = 0;
    jackpot = 1 ether;
    lastInvestmentTimestamp = 0;}
}

```

(b) Timestamp-dependency false-negative case.

```solidity
contract TimestampMisleading {

    function compute() public view returns
    (string memory) {
        uint x = block.timestamp;
        uint y = uint((4 * x) % 2);
        if (15 <= y && y >= 20) {
            // Critical section
            return "Entered critical section";
        } else {
            return "Condition not met";}
    }
}

```

(c) Timestamp-dependency false-positive case.

```solidity
contract EGame {
address payable private winner ;
uint startTime ;

constructor () public {
winner = msg.sender ;
startTime = block.timestamp ;}

function play (bytes32 guess) public {
if (keccak256(abi.encode(guess))==keccak256
(abi.encode('solution'))){
if (startTime + (5 * 1 days )==block.timestamp) {
winner = msg.sender;}}}

function getReward () payable public {
winner.transfer(msg.value);} }

```

(d) Transaction-ordering-dependency false-negative case.

Fig. 2. Representative false-positive and false-negative cases in reentrancy, timestamp dependency, and transaction-ordering dependency detection.

Just Accepted

2 Why Semantics Matter in SCVD

Blockchain provides a decentralized, untrusted execution environment for smart contracts. Solidity, as a domain-specific smart contract language for Ethereum, presents unique constructs that distinguish it from traditional programming languages. Unlike traditional software systems, smart contracts operate over blockchain-specific mutable states, including persistent storage and account balances. This results in a new kind of vulnerabilities that cannot be adequately modeled without considering both the local execution context and the underlying blockchain state. However, our analysis reveals that most of the limitations in existing tools are due to reliance on certain syntactic patterns, such as Check-Effect-Interaction in the case of reentrancy, and a lack of sound semantics approximation [49].

Figure 2 presents representative cases in which existing tools produce either false positives or false negatives. Figure 2a shows the `Reentrancy_safe` contract. The contract clears the caller's recorded reward before executing the external call. Therefore, if the callee re-enters `withdrawReward()`, the new invocation reads zero into `amountToWithdraw` and is stopped by `require(amountToWithdraw > 0)`. The recorded reward can consequently be transferred only once. The latest versions of Mythril and Oyente+ report reentrancy for this safe contract, whereas our analysis reports no reentrancy. This is because the potential reentrant execution is semantically infeasible after the caller's recorded reward is reset to zero before the external call. This example shows that the presence of an external call alone is insufficient to establish a reentrancy vulnerability; the semantic feasibility of the reentrant execution must be determined from the contract state at the call site. Figure 2b shows the *Governmental* contract, which contains a timestamp dependence vulnerability at line 17. A timestamp dependency vulnerability arises when a contract uses a miner-influenced blockchain parameter, typically `block.timestamp` to make security critical decisions. Since Ethereum miners can adjust the block timestamp within a bounded range, an adversarial miner can manipulate the outcome of such decisions. For example, the Governmental contract implements a simple investment game where users invest at least half the current jackpot, and if no new investment arrives within one minute, the last investor claims the jackpot. Observe that line 17 (`lastInvestmentTimestamp = block.timestamp`) enables a miner to manipulate the one-minute window in `resetInvestment()`. The latest versions of Mythril and SmartCheck do not report this timestamp dependency vulnerability. For this contract, Mythril does not track the timestamp value stored in `lastInvestmentTimestamp` from `invest()` to `resetInvestment()`. Therefore, it does not connect the assignment of `block.timestamp` with the later time-based check and does not report the vulnerability. SmartCheck does not report this case because, in the evaluated implementation, its XPath-based timestamp-dependence rule detects the legacy `now` keyword, whereas the `Governmental` contract uses `block.timestamp`. To further understand the semantic dependencies of these tools, we examine the contract in Figure 2c. We observe a syntactic dependency of variable *y* on *block.timestamp* through *x*. However, closer analysis reveals that the expression y = (4 * x) % 2 always evaluates to zero for any integer input, indicating that the dependency is semantically irrelevant. Still, both Mythril and Slither wrongly label this contract as timestamp-dependent. Lines 9-16 of Figure 2d illustrate the EGame contract, which contains a TOD vulnerability. In this, the contract declares a game winner based on the order in which the transaction occurs. Malicious actors exploit this by manipulating the sequence of two transactions within a block, altering the outcome. Mythril and Oyente+ do not report this vulnerability. For this contract, their analyses do not connect the update of `winner` in `play()` with its later use in `getReward()` to determine who receives the Ether under different transaction orders.

These limitations reveal a broader shortcoming: the lack of a compositional, semantics-aware framework that can precisely capture smart contracts' blockchain-specific execution semantics to reason about these possible vulnerable behaviours. This motivates our work, which integrates formal abstract semantics with domain-sensitive fixpoint reasoning to detect, with improved precision, vulnerabilities that syntactic or bounded symbolic methods often miss.

Just Accepted

3 Formal Syntax and Concrete Semantics of Solidity Language

In this section, we define the formal syntax and semantics of a core subset of Solidity, establishing a theoretical foundation for precise SCVD analysis. This formalization is not intended to be exhaustive; rather, it captures the essential language constructs needed for the formal development. In the implementation, we additionally support a broader range of Solidity constructs beyond those included in the formal core, ensuring practical coverage.

3.1 Solidity Syntax

Table 1. An Abstract syntax modeling subset of Solidity

Expressions			
exp	$\in$	E	Arithmetic Expressions
exp	$::=$	v	Numerical values $\in \mathbb{R}$
	$\mid$	id	Variable
	$\mid$	$\text{exp}_1 \oplus \text{exp}_2$	where $\oplus \in \{+, -, *, /\}$
bexp	$\in$	B	Boolean Expressions
bexp	$::=$	*true* $\mid$ *false*	Truth/Falsity
	$\mid$	$\text{exp}_1 \oslash \text{exp}_2$	$\oslash \in \{\geqslant, \leqslant, <, >, ==\}$
	$\mid$	$\text{bexp}_1 \otimes \text{bexp}_2$	$\otimes \in \{\wedge, \vee\}$
	$\mid$	$\neg$bexp	Negation
Types and Storage			
type	$::=$	*int* $\mid$ *uint* $\mid$ *bool*	
	$\mid$	*address* $\mid$ *address payable*	Value Types
loc	$::=$	*storage* $\mid$ *memory*	Data Location
Statements and Contracts			
sdecl	$::=$	type id ($:=$ exp)?;	State Variable Declaration
SV	$::=$	sdecl*	Collection of State Variables
ldecl	$::=$	type loc id ($:=$ exp)?;	Local Declaration
assgn	$::=$	id $:=$ exp;	Assignment
par	$::=$	type loc? id $\mid$ par_1, par_2	Function Parameter
constr	$::=$	*constructor*(par?) {stmt?}	Default/Explicit Constructor
func	$::=$	*function* id(par?) {stmt}	Function Definition without Return
	$\mid$	*function* id(par?) *returns* (par) {stmt}	Function Definition with Return
	$\mid$	*fallback*() {stmt}	Fallback Function Definition
Fcall	$::=$	func; $\mid$ (par) $:=$ func;	Function Call
stmt	$::=$	ldecl $\mid$ assgn $\mid$ Fcall	Statement
	$\mid$	*if* bexp *then* stmt_1 *else* stmt_2	Conditional
	$\mid$	*while* bexp *do* stmt *done*	Loop
	$\mid$	stmt_1 stmt_2	Sequence
sc	$::=$	$\langle$SV, constr, func$^+\rangle$	Smart contract
States and Environments			
E_L	$::=$	$\texttt{Vars}_l \mapsto \texttt{MemLoc}$	Local Environment Mapping
E_B	$::=$	$\texttt{Vars}_b \mapsto \texttt{BLoc}$	Blockchain Environment Mapping
S_L	$::=$	$\texttt{MemLoc} \mapsto \texttt{Val}$	Local Stores
S_B	$::=$	$\texttt{BLoc} \mapsto \texttt{Val}$	Blockchain Stores
Σ_l	$::=$	$E_L \times S_L$	Local States
Σ_b	$::=$	$E_B \times S_B$	Blockchain States
Σ	$::=$	$\Sigma_l \times \Sigma_b$	Solidity States

Just Accepted

As Ethereum's foremost smart contract language, Solidity encompasses both general-purpose programming constructs and blockchain-specific properties. Table 1 defines the abstract syntax of a subset of Solidity constructs, covering expressions, declarations, constructor, functions and smart contract definitions. Arithmetic expressions exp comprise program variables id, numerical values v, and binary arithmetic operations $\oplus$ over different arithmetic expressions. Boolean expressions bexp comprise relational operators $\oslash$ over arithmetic expressions or

logical connectives ⊗ over boolean expressions, which result a truth value. The abstract syntax for value types, assignment, sequencing, conditionals and loops follow the standard form of imperative languages [50].

To capture Solidity constructs specific to blockchain, we introduce state variable declarations sdecl which define persistent contract variables. The collection of all state variable declarations of a contract is represented by SV. Within a function definition func, both parameters par and local declarations ldecl introduce variables by specifying a data-location qualifier loc ∈ { storage, memory }, indicating the type of memory where the variable's storage is allocated. Observe that the variables under SV are associated with blockchain storage only, whereas the variable under local declaration ldecl may associate with either blockchain storage or volatile memory according to its loc qualifier. This distinction determines whether a write to the variable persists on the blockchain after the transaction ends or is discarded with it.

A constructor constr is executed exactly once during contract deployment and is responsible to initialize state variables. We define syntax of the explicitly specified constructor constr in Solidity, which considers optional parameter list and a set of statements stmt responsible to initialize state variables. Observe that, in the case where constructor is not explicitly defined in the contract, a default constructor *constructor*(){} is executed to initialize all state variables to their default values. We define abstract syntax of functions func that take an optional list of parameters par, capturing both the functions that return values and functions that do not, with their corresponding forms of function invocation represented by Fcall. Observe that since Solidity functions may return multiple values, our syntax includes tuple assignment of the form (par) := func to model the binding of returned values. Function bodies may contain local declarations, assignments, function calls, conditional statements, iteration constructs, and sequential composition. Finally, a smart contract sc is modelled as the tuple $\langle \mathsf{SV}, \mathsf{constr}, \mathsf{func}^+ \rangle$ of its collection of state variables, its constructor, and its collection of function definitions, defined below.

Definition 3.1 (Smart Contract sc*).* A Solidity smart contract sc is modeled as a tuple $\langle \mathsf{SV}, \mathsf{constr}, \mathsf{func}^+ \rangle$, where SV is the collection of state variables, constr is the contract constructor, and func^+ is the collection of function definitions.

3.2 Formalizing Concrete Semantics

In this section, we define formal semantics of Solidity constructs presented in Table 1. In the rest of the section, we denote the set of functions as $[D \mapsto R] ::= \{f \mid f \text{ is a function such that } \mathsf{dom}(f) = D \text{ and } \mathsf{range}(f) \subseteq R\}$.

A distinguishing feature of Solidity is that contracts operate over two kinds of memory. Main memory is temporary and is discarded when a transaction ends, whereas blockchain storage is permanent and persists across transactions. To make this distinction explicit, we split a Solidity state into a local part and a blockchain part, defined as follows: Given the set of variables Vars, let $\mathtt{Vars}_l$ and $\mathtt{Vars}_b$ be the sets of local and blockchain variables, respectively. Each variable in $\mathtt{Vars}_l$ is mapped to an address in local memory MemLoc, while each variable in $\mathtt{Vars}_b$ is mapped to an address in blockchain memory BLoc. As defined in Table 1, we define the set of local environments $E_L : [\mathtt{Vars}_l \mapsto \mathtt{MemLoc}]$ and the set of blockchain-environment $E_B : [\mathtt{Vars}_b \mapsto \mathtt{BLoc}]$ in terms of the mapping of variables to their corresponding memory locations. Similarly, we define the set of local stores $S_L : [\mathtt{MemLoc} \mapsto \mathtt{Val}]$ and the set of blockchain stores $S_B : [\mathtt{BLoc} \mapsto \mathtt{Val}]$ which represent mapping of respective memory locations to the domain values in Val. To define the semantics, we outline a set of states as follows: (i) $\Sigma_l = E_L \times S_L$ is the set of local states (ii) $\Sigma_b = E_B \times S_B$ is the set of blockchain states (iii) $\Sigma = \Sigma_l \times \Sigma_b$ denote the set of Solidity states.

A state $\sigma \in \Sigma$ is written as a tuple $\langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b \rangle$ where $\mathbf{e}_l \in E_L$, $\mathbf{s}_l \in S_L$, $\mathbf{e}_b \in E_B$ and $\mathbf{s}_b \in S_B$. There exist several special variables in Solidity that have special meaning. Given a state σ, their semantics are defined as follows:

Just Accepted

- Ethereum follows an account-based model, where an address on the blockchain uniquely identifies either user's or smart contract's account. Given an address Addr of the user or smart contract, we denote by σ[Addr](bal) the balance of the account corresponding to Addr.
- σ(block.timestamp) represents the block's timestamp that includes the current transaction under state σ.
- σ(now) is an alias for σ(block.timestamp).
- σ(msg.sender) identifies the address of the external account or contract that invoked the current function call under state σ.
- σ(msg.value) gives the amount of Ether (in Wei) sent along with the current message or transaction under state σ.
- σ(block.number) denotes the height of the current block within the blockchain under state σ.
- σ(gasleft) returns the remaining gas available for the execution of the current transaction under state σ.

We define three semantic functions: (1) $\mathscr{S}_e : E \mapsto (\Sigma \mapsto \mathbb{R})$ which specifies the meaning of arithmetic expressions by mapping each arithmetic expression exp $\in E$ to a function that evaluates it in a given state $\sigma \in \Sigma$ and produces a numerical value $v \in \mathbb{R}$, (2) $\mathscr{S}_b : B \mapsto (\Sigma \mapsto \{true, false\})$ which specifies the meaning of boolean expressions by mapping each boolean expression bexp $\in B$ to a function that evaluates it in a given state $\sigma \in \Sigma$ and produces a truth value $t \in \{true, false\}$, and (3) $\mathscr{S}_c : P \mapsto (\Sigma \mapsto \Sigma)$ which specifies the meaning of Solidity statements by mapping each statement stmt $\in P$ to a function that evaluates it in a given state $\sigma \in \Sigma$ and produces a new state $\sigma' \in \Sigma$.

Expressions. While we follow standard semantic definitions for Arithmetic expressions [50], the crucial Solidity-specific point is in the semantics for id. Given a state $\sigma = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b \rangle$, the lookup uses the local store $\mathbf{s}_l$ when the variable lives in memory and the blockchain store $\mathbf{s}_b$ when it lives in storage. Accordingly, we define the semantics of arithmetic and boolean expressions of Solidity below.

$$\mathscr{S}_e[\![v]\!] = \{\, (\sigma, v) \mid \sigma \in \Sigma \,\}$$

$$\mathscr{S}_e[\![\mathsf{id}]\!] = \begin{cases} \{\, (\sigma, v) \mid \sigma \in \Sigma,\ \mathbf{s}_l(\mathbf{e}_l(\mathsf{id})) = v \,\} & \text{if } \mathbf{e}_l(\mathsf{id}) \in \mathtt{MemLoc} \\ \{\, (\sigma, v) \mid \sigma \in \Sigma,\ \mathbf{s}_b(\mathbf{e}_b(\mathsf{id})) = v \,\} & \text{if } \mathbf{e}_b(\mathsf{id}) \in \mathtt{BLoc} \end{cases}$$

$$\mathscr{S}_e[\![\mathsf{exp}_1 \oplus \mathsf{exp}_2]\!] = \{\, (\sigma, v_1 \oplus v_2) \mid (\sigma, v_1) \in \mathscr{S}_e[\![\mathsf{exp}_1]\!],\ (\sigma, v_2) \in \mathscr{S}_e[\![\mathsf{exp}_2]\!] \},\ \oplus \in \{+, -, \times\} \vee (\oplus \in \{/, \%\} \wedge v_2 \neq 0)$$

$$\mathscr{S}_b[\![\mathsf{exp}_1 \oslash \mathsf{exp}_2]\!] = \{\, (\sigma, v_1 \oslash v_2) \mid (\sigma, v_1) \in \mathscr{S}_e[\![\mathsf{exp}_1]\!],\ (\sigma, v_2) \in \mathscr{S}_e[\![\mathsf{exp}_2]\!] \}, \oslash \in \{\geqslant, \leqslant, <, >, ==\}$$

$$\mathscr{S}_b[\![true]\!] = \{\, (\sigma, true) \mid \sigma \in \Sigma \,\},\ \mathscr{S}_b[\![false]\!] = \{\, (\sigma, false) \mid \sigma \in \Sigma \,\}.$$

$$\mathscr{S}_b[\![\mathsf{bexp}_1 \otimes \mathsf{bexp}_2]\!] = \{\, (\sigma, b_1 \otimes b_2) \mid (\sigma, b_1) \in \mathscr{S}_b[\![\mathsf{bexp}_1]\!],\ (\sigma, b_2) \in \mathscr{S}_b[\![\mathsf{bexp}_2]\!] \}, \otimes \in \{\vee, \wedge\}$$

$$\mathscr{S}_b[\![\neg \mathsf{bexp}]\!] = \{\, (\sigma, \neg b) \mid (\sigma, b) \in \mathscr{S}_b[\![\mathsf{bexp}]\!] \,\}$$

Local Variable Declaration. Given Solidity states $\sigma = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b \rangle \in \Sigma$, the semantics of local variable declaration is defined as:

$$\mathscr{S}_c[\![\mathsf{type\ loc\ id}]\!] \triangleq \begin{cases} \{\sigma \to \sigma' \mid \sigma \in \Sigma\} & \text{if loc} = \textit{memory} \\ \{\sigma \to \sigma'' \mid \sigma \in \Sigma\} & \text{if loc} = \textit{storage} \end{cases}$$

where $\sigma' = \langle \mathbf{e}_l[\mathsf{id} \to a^l_{in}], \mathbf{s}_l[a^l_{in} \to v'_d], \mathbf{e}_b, \mathbf{s}_b \rangle$ with $a^l_{in} \in$ MemLoc as a new local memory location with the default memory value v'_d, and $\sigma'' = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b[\mathsf{id} \to a^s_{in}], \mathbf{s}_b[a^s_{in} \to v''_d] \rangle$ with $a^s_{in} \in$ BLoc as a newly allotted storage location with the default storage value v''_d. Here, the loc qualifier decides where the new variable is placed. Accordingly, a *memory* variable is allocated in MemLoc and a *storage* variable is allocated in BLoc, and they are initialized with default values. This leads to a change in the corresponding environment and store of the state and results in a new state. Similarly, when the variable is initialized at the time of its declaration, the semantics is defined as:

Just Accepted

$$\mathscr{S}_c[\![\text{type loc id } := \text{exp}]\!] \triangleq \begin{cases} \{\sigma \to \sigma' \mid \sigma \in \Sigma\} & \text{if loc} = \textit{memory} \\ \{\sigma \to \sigma'' \mid \sigma \in \Sigma\} & \text{if loc} = \textit{storage} \end{cases}$$

where $(\sigma, v) \in \mathscr{S}_e[\![\text{exp}]\!]$, $\sigma' = \langle \mathbf{e}_l[\text{id} \to a^l_{in}], \mathbf{s}_l[a^l_{in} \to v], \mathbf{e}_b, \mathbf{s}_b \rangle$ with $a^l_{in} \in \mathsf{MemLoc}$ as a fresh local memory location, and $\sigma'' = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b[\text{id} \to a^s_{in}], \mathbf{s}_b[a^s_{in} \to v] \rangle$ with $a^s_{in} \in \mathsf{BLoc}$ as a fresh storage location.

Assignment Statement. In case of assignment, the store corresponding to the left hand side variable id will be updated by the value obtained by evaluating the right hand side arithmetic expression. The update goes to the local store when id is a memory variable and to the blockchain store when it is a storage variable. Accordingly, we define the semantics of the assignment statement below.

$$\mathscr{S}_c[\![\text{id } := \text{exp}]\!] \triangleq \begin{cases} \{\sigma \to \sigma' \mid \sigma \in \Sigma\} & \text{if } \mathbf{e}_l(\text{id}) \in \mathsf{MemLoc} \\ \{\sigma \to \sigma'' \mid \sigma \in \Sigma\} & \text{if } \mathbf{e}_b(\text{id}) \in \mathsf{BLoc} \end{cases}$$

where $\sigma = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b \rangle$, $(\sigma, v) \in \mathscr{S}_e[\![\text{exp}]\!]$, $\sigma' = \langle \mathbf{e}_l, \mathbf{s}_l[\mathbf{e}_l(\text{id}) \to v], \mathbf{e}_b, \mathbf{s}_b \rangle$, $\sigma'' = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b[\mathbf{e}_b(\text{id}) \to v] \rangle$.

Sequence. The semantics of sequential composition is defined as the relational composition of the semantics of the individual statements. A state σ'' is reachable from a state σ if there exists an intermediate state σ' such that σ' is reachable from σ through the execution of stmt_1, and σ'' is reachable from σ' through the execution of stmt_2. Formally, this is defined as: $\mathscr{S}_c[\![\text{stmt}_1\ \text{stmt}_2]\!] \triangleq \{\sigma \to \sigma'' \mid \sigma \in \Sigma,\ \sigma \to \sigma' \in \mathscr{S}_c[\![\text{stmt}_1]\!], \sigma' \to \sigma'' \in \mathscr{S}_c[\![\text{stmt}_2]\!]\}$

Conditional. The semantics of a conditional statement is defined by selecting the semantics of the appropriate branch according to the evaluation of the boolean expression in the current state.

$$\begin{aligned} \mathscr{S}_c[\![\text{if bexp then stmt}_1 \text{ else stmt}_2]\!] \triangleq & \{\sigma \to \sigma' \mid \sigma \in \Sigma,\ (\sigma, \text{true}) \in \mathscr{S}_b[\![\text{bexp}]\!],\ (\sigma \to \sigma') \in \mathscr{S}_c[\![\text{stmt}_1]\!]\} \\ & \cup \{\sigma \to \sigma'' \mid \sigma \in \Sigma,\ (\sigma, \text{false}) \in \mathscr{S}_b[\![\text{bexp}]\!],\ (\sigma \to \sigma'') \in \mathscr{S}_c[\![\text{stmt}_2]\!]\} \end{aligned}$$

Iteration. The semantics of the iterative statements is defined in terms of fixed-point solutions, as shown below.
$\mathscr{S}_c[\![\text{while bexp do stmt}]\!] \triangleq \text{lfp}^{\subseteq}_{\emptyset}(F)$,where

$$\begin{aligned} F(\psi) = & \{\sigma \to \sigma' \mid \sigma \in \Sigma,\ (\sigma, \text{true}) \in \mathscr{S}_b[\![\text{bexp}]\!],\ (\sigma \to \sigma') \in \mathscr{S}_c[\![\text{stmt}]\!]\} \\ & \cup\ \{\sigma \to \sigma \mid \sigma \in \Sigma,\ (\sigma, \text{false}) \in \mathscr{S}_b[\![\text{bexp}]\!]\} \\ & \cup\ \{\sigma \to \sigma'' \mid \sigma \in \Sigma,\ (\sigma, \text{true}) \in \mathscr{S}_b[\![\text{bexp}]\!],\ (\sigma \to \sigma') \in \mathscr{S}_c[\![\text{stmt}]\!],\ (\sigma' \to \sigma'') \in \psi\} \end{aligned}$$

Here, $\text{lfp}(F)$ denotes the least fixpoint of the monotonic function F. Starting from the empty set $\emptyset$, F is applied repeatedly. When F is applied first time, it collects every $(\sigma \to \sigma')$ pair from running the loop body once. Similarly, it includes 'no-change' behavior $(\sigma \to \sigma)$ when the execution exits from loop due to unsatisfiability of the condition bexp. This also includes all other behaviors $(\sigma \to \sigma'')$ which are generated from multiple loop iterations. When an application of F produces no new pairs, the process stops. The final set contains exactly the state transitions for zero, one, two, and so on, up to every possible number of loop iterations.

Constructor. Let $\text{sc} = \langle \text{SV}, \text{constr}, \text{Func} \rangle$ denote a smart contract, where constr is defined as constructor (par){stmt}. Let $v_{in} \in \text{Val}$ represent the input values, and $\sigma_0 = \langle \mathbf{e}_{l0}, \mathbf{s}_{l0}, \mathbf{e}_{b0}, \mathbf{s}_{b0} \rangle$ be the initial state at the point of contract deployment. Let $a^l_{in} \in \mathsf{MemLoc}$ and $a^b_{in} \in \mathsf{BLoc}$ denote freshly allocated locations in local memory and blockchain storage, respectively. The semantics of the contract constructor are formalized as follows:

$$\mathscr{S}_c[\![\textit{constructor}(\text{par})\{\text{stmt}\}]\!] \triangleq \left\{ \left((\sigma_0, v_{in}) \to \sigma''\right) \mid \sigma_0 \in \Sigma \right\}$$

where $\sigma' = \langle \mathbf{e}_{l0}[\text{Par} \mapsto a^l_{in}],\ \mathbf{s}_{l0}[a^l_{in} \mapsto v_{in}],\ \mathbf{e}_{b0}[\text{SV} \mapsto a^b_{in}],\ \mathbf{s}_{b0} \rangle$ and $(\sigma' \to \sigma'') \in \mathscr{S}_c[\![\text{stmt}]\!]$. If the constructor does not accept any parameter par, the corresponding local environment $\mathbf{e}_{l0}$ and store $\mathbf{s}_{l0}$ remain unchanged. If no constructor is explicitly present in the contract, a default constructor is executed. The semantics for a default

Just Accepted

constructor (denoted constr) is given by:

$$\mathscr{S}_c[\![\mathit{constructor}()\{\}]\!] \triangleq \{\, (\sigma_0 \to \sigma') \mid \sigma_0 \in \Sigma \,\}$$

where $\sigma' = \langle \mathbf{e}_{l0}, \mathbf{s}_{l0}, \mathbf{e}_{b0}\,[\mathsf{SV} \mapsto a^s_{in}],\ \mathbf{s}_{b0}\,[a^s_{in} \mapsto v_d]\rangle$ with $a^s_{in} \in \mathsf{BLoc}$ a fresh storage location for storage variables SV which are initialized to the default values v_d.

Function. Let $\mathsf{Par}^{\mathsf{memory}}_{\mathsf{in}}$ and $\mathsf{Par}^{\mathsf{storage}}_{\mathsf{in}}$ denote the sets of function parameters allocated in memory and blockchain storage, respectively. Assume v^{memory}_{in} and $v^{storage}_{in}$ be the collections of input values related to $\mathsf{Par}^{\mathsf{memory}}_{\mathsf{in}}$ and $\mathsf{Par}^{\mathsf{storage}}_{\mathsf{in}}$, so that $v_{in} = (v^{memory}_{in} \cup v^{storage}_{in}) \in \mathsf{Val}$. The semantics of a function are expressed by:

$$\mathscr{S}_c[\![\mathit{function}\ \mathsf{id_f}\,(\mathsf{par_{in}})\ (\mathit{returns}\ \mathsf{par_{out}})\,\{\,\mathsf{stmt}\,\}]\!] \triangleq \{\, ((\sigma, v_{in}) \to (\sigma'', v_{out})) \mid \sigma \in \Sigma \,\}$$

where $\sigma = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b\rangle, \sigma' = \langle \mathbf{e}_l\,[\mathsf{Par}^{\mathsf{memory}}_{\mathsf{in}} \mapsto a^l_{in}],\ \mathbf{s}_l\,[a^l_{in} \mapsto v^{memory}_{in}],\ \mathbf{e}_b\,[\mathsf{Par}^{\mathsf{storage}}_{\mathsf{in}} \mapsto a^s_{in}],\ \mathbf{s}_b\,[a^s_{in} \mapsto v^{storage}_{in}]\rangle$ with $a^l_{in} \in \mathsf{MemLoc}$ and $a^s_{in} \in \mathsf{BLoc}$ are fresh memory and storage locations, and $(\sigma' \to \sigma'') \in \mathscr{S}_c[\![\mathsf{stmt}]\!]$ and $(\sigma'', v_{out}) \in \mathscr{S}_c[\![\mathsf{par_{out}}]\!]$. Here, each input is bound to a fresh memory locations a^l_{in} or fresh storage locations a^s_{in} according to the data location of the parameter. The body stmt then executes, giving the new state σ'' and the return value v_{out}. In case of function without parameter, the corresponding local environment and store in σ remains unchanged. Similarly, in case of function without a return statement, the corresponding v_{out} will be absent in the semantic definition. Observe that the semantics of the fallback function is defined in a similar way.

Function Call with Return. Given the function call statement $(\mathsf{par}) := \mathsf{func}$. Let a^l and a^s, respectively, be the memory locations and storage locations of the variables in par. Accordingly, let v^{memory}_{out} and $v^{storage}_{out}$ be the collections of output values returned by the function func, which are to be assigned to a^l and a^s respectively. We define the semantics of a function call as follows:

$$\mathscr{S}_c[\![(\mathsf{par}) := \mathsf{func}]\!] \triangleq \begin{cases} \{\, \sigma \to \sigma'' \mid \sigma \in \Sigma,\ ((\sigma, v_{in}) \to (\sigma', v_{out})) \in \mathscr{S}_c[\![\mathsf{func}]\!] \,\} & \text{if func} \in \mathsf{sc} \\ \mathscr{S}_c[\![\mathit{fallback}()\ \{\mathsf{stmt}\}]\!] & \text{otherwise} \end{cases}$$

where $\sigma' = \langle \mathbf{e}'_l, \mathbf{s}'_l, \mathbf{e}'_b, \mathbf{s}'_b\rangle$ and $\sigma'' = \langle \mathbf{e}'_l, \mathbf{s}'_l\,[a^l \mapsto v^{memory}_{out}],\ \mathbf{e}'_b,\ \mathbf{s}'_b\,[a^s \mapsto v^{storage}_{out}]\rangle$. Observe that, in the case of a call to a non-existent function, the contract semantics is defined in terms of the semantics of the intra-contract fallback function without function name. The semantics of the fallback function is defined in the same way as that of any other function without return values.

Example 3.2. Consider the code snippet comprising the following three statements.

$$\mathsf{uint\ storage\ b} := 100; \quad \mathsf{uint\ memory\ x} := 20; \quad \mathsf{if\ (b} \geqslant \mathsf{x)\ then\ b} := \mathsf{b} - \mathsf{x\ else\ x} := 0;$$

Let $\sigma_0 = \langle \mathbf{e}_l, \mathbf{s}_l, \mathbf{e}_b, \mathbf{s}_b\rangle$ be the initial state. The first declaration allocates a fresh storage location $a_1 \in \mathsf{BLoc}$ and assigns 100 to a_1. This leads to a change of the state $\sigma \mapsto \sigma'$ where $\sigma' = \langle \mathbf{e}_l,\ \mathbf{s}_l,\ \mathbf{e}_b\,[\mathsf{b} \mapsto a_1],\ \mathbf{s}_b\,[a_1 \mapsto 100]\rangle$. The second one allocates a fresh memory location $a_2 \in \mathsf{MemLoc}$ and assigns 20 to it. This results the state change $\sigma' \mapsto \sigma''$ where $\sigma'' = \langle \mathbf{e}_l\,[\mathsf{x} \mapsto a_2],\ \mathbf{s}_l\,[a_2 \mapsto 20],\ \mathbf{e}_b\,[\mathsf{b} \mapsto a_1],\ \mathbf{s}_b\,[a_1 \mapsto 100]\rangle$. For the conditional, as the semantics of the condition $\mathscr{S}_b[\![\mathsf{b} \geqslant \mathsf{x}]\!]$ yields $(100 \geqslant 20) = \mathit{true}$, so the then-branch runs. This leads the state change $\sigma'' \mapsto \sigma'''$ where $\sigma''' = \langle \mathbf{e}_l\,[\mathsf{x} \mapsto a_2],\ \mathbf{s}_l\,[a_2 \mapsto 20],\ \mathbf{e}_b\,[\mathsf{b} \mapsto a_1],\ \mathbf{s}_b\,[a_1 \mapsto 80]\rangle$, as the semantics of arithmetic expression $b - x$ on the right hand side of assignment lead to $(\sigma'', 80) \in \mathscr{S}_e[\![\mathsf{b} - \mathsf{x}]\!]$ and the cell a_1 corresponding to the left hand side variable b of the assignment is overwritten.

Just Accepted

Smart Contract Fixpoint Semantics. The semantics of a smart contract are characterized in terms of traces, each representing a possible sequence of states describing the contract's evolution after deployment on the blockchain. This is defined in terms of the fixpoint semantics, as follows.

Definition 3.3 (Smart Contract Fixpoint Semantics). Let $\mathsf{sc} = \langle \mathsf{SV},\ \mathsf{constr},\ \mathsf{Func} \rangle$. Given an initial state σ_0 at the moment of contract deployment, the semantics of sc are defined as:

$$\mathscr{S}_c[\![\mathsf{sc}]\!] = \mathrm{lfp}^{\subseteq}_{\emptyset}\ \lambda T.\ \mathbb{I}_0\ \cup \left\{ \sigma_0 \xrightarrow{\hat{\ell}_0} \ldots \xrightarrow{\hat{\ell}_{n-1}} \sigma_n \xrightarrow{(\mathsf{func}, v_{in})} \sigma_{n+1} \;\middle|\; \sigma_0 \to \ldots \to \sigma_n \in T,\ (\sigma_{n+1}, v_{out}) \in \mathscr{S}_c[\![\mathsf{func}]\!](\sigma_n, v_{in}),\ \mathsf{func} \in \mathsf{Func} \right\}$$

where $\mathbb{I}_0 = \{\, \mathscr{S}[\![\mathsf{constr}]\!](\sigma_0, v_{in}) \mid v_{in} \in \mathsf{Val} \}$ and each transition label $\hat{\ell}_i$ is drawn from $(\mathsf{Func} \times \mathsf{Val})$.

Here, $\mathrm{lfp}^{\subseteq}_{\emptyset} F$, where the monotonic function F is expressed as a lambda expression, denotes the least fixed-point solution starting from the set of input traces $T = \emptyset$. In the first application of F, it collects $\mathbb{I}_0$, which contains the states produced by the constructor when the contract is deployed. In each subsequent application, new traces are generated by executing the functions in an arbitrary order (determined by the transaction ordering chosen by the miners) from the last states of the traces accumulated in T thus far. This process continues until no further changes are observed, indicating that a fixed point has been reached. The resulting set of all possible traces obtained by executing the contract from its deployment onward constitutes the trace semantics of the contract.

Definition 3.4 (Reachable States Semantics). The reachable states semantics of the smart contract sc in the fixpoint form is defined as: $\mathscr{S}_c[\![\mathsf{sc}]\!] = \mathrm{lfp}^{\subseteq}_{\emptyset}\ \lambda S.\ \mathbb{I}_0 \cup \bigcup_{\mathsf{func} \in \mathsf{Func}} \mathscr{S}_c[\![\mathsf{func}]\!](S)$ where $\mathbb{I}_0 = \{\, \mathscr{S}_c[\![\mathsf{Constr}]\!](\sigma_0, v_{in}) \mid v_{in} \in \mathsf{Val} \}$ and $\mathscr{S}_c[\![\mathsf{func}]\!](S) = \left\{ \sigma' \mid \sigma \in S,\ v_{in} \in \mathsf{Val},\ (\sigma', v_{out}) \in \mathscr{S}_c[\![\mathsf{func}]\!](\sigma, v_{in}) \right\}$.

Observe that, unlike the trace semantics, this semantic definition considers only the states that are reachable during the execution of the contract since its deployment.

4 Sound Approximation of Solidity Semantics

We first briefly recall the Abstract Interpretation theory [11, 12]. Then, we define the abstract semantics of Solidity in terms of various non-relational and relational abstract domains, which establish the foundation for sound static analysis of Solidity Contracts.

4.1 The Abstract Interpretation Theory

Abstract Interpretation is a method for soundly approximating the concrete semantics of a program, which enables obtaining sound answers about how the program will behave at runtime. The concept involves elevating concrete semantics to an abstract domain by substituting concrete values with abstract values (representing properties of interest) and by replacing concrete operations with their abstract counter-part over the abstract values.

Typically, the mapping between the concrete domain and the abstract domain is formalized through a Galois Connection which involves two monotonic functions, namely an abstraction function α and a concretization function γ. The abstraction function α maps concrete values to their abstract counterparts, while the concretization function γ maps abstract values to their concrete counter-parts. Galois Connection guarantees that reasoning performed in the abstract domain soundly approximates reasoning in the concrete domain. Formally, it is defined below.

Definition 4.1 (Galois Connection [11]). Given the concrete domain $\langle \mathbb{S}, \sqsubseteq \rangle$ and the abstract domain $\langle \overline{\mathbb{S}}, \overline{\sqsubseteq} \rangle$, both structured as partially ordered sets where the ordering relations indicate the precision levels of the elements. A Galois connection between these two domains is a quadruple $\langle (\mathbb{S}, \sqsubseteq), \alpha, \gamma, (\overline{\mathbb{S}}, \overline{\sqsubseteq}) \rangle$, where $\alpha : \mathbb{S} \to \overline{\mathbb{S}}$ (abstraction function) and $\gamma : \overline{\mathbb{S}} \to \mathbb{S}$ (concretization function) are monotonic functions satisfying: (i) $\forall x \in \mathbb{S}.\ x \sqsubseteq \gamma(\alpha(x))$

Just Accepted

(extensiveness), and (ii) $\forall \overline{x} \in \overline{\mathbb{S}}.\ \alpha(\gamma(\overline{x}))\overline{\sqsubseteq}\overline{x}$ (reductiveness). Equivalently, $\langle(\mathbb{S}, \sqsubseteq), \alpha, \gamma, (\overline{\mathbb{S}}, \overline{\sqsubseteq})\rangle$ forms a Galois connection iff $\forall x \in \mathbb{S},\ \forall \overline{x} \in \overline{\mathbb{S}}.\ \alpha(x)\overline{\sqsubseteq}\overline{x} \iff x \sqsubseteq \gamma(\overline{x})$

Choosing an appropriate abstract domain that strikes the best balance between expressive power and computational efficiency is a significant challenge in abstract interpretation. Figure 3 shows a comparative analysis of three widely used abstract domains in terms of precision and computational cost. The *interval domain* [11] is a non-relational abstract domain that maps each program variable to a closed interval without preserving any relationships between them. Relational domains improve this analysis by representing the interactions between variables. The *octagon domain* [32] represents pairwise constraints in the form of $\pm x_i \pm x_j \leq c$, which allow the analysis within polynomial time. On the other hand, the *polyhedra domain* [13] models arbitrary linear inequalities among variables, achieving better precision but at substantially increased exponential computational cost.

To overapproximate the concrete semantics of the Solidity subset in the abstract domain of $\mathcal{D}$, let $\overline{\Sigma} = \overline{\Sigma}_l \times \overline{\Sigma}_b$ be the set of abstract states, where $\overline{\Sigma}_l = \overline{E}_L \times \overline{S}_L$ is the set of abstract local states and $\overline{\Sigma}_b = \overline{E}_B \times \overline{S}_B$ is the set of abstract blockchain states. This state abstraction is sound, forming the Galois connection $\langle \Sigma, \alpha, \gamma, \overline{\Sigma}\rangle$ such that: $\forall\, \sigma \in \Sigma,\ \forall\, \overline{\sigma} \in \overline{\Sigma} : \alpha(\sigma) \sqsubseteq \overline{\sigma} \iff \sigma \subseteq \gamma(\overline{\sigma})$

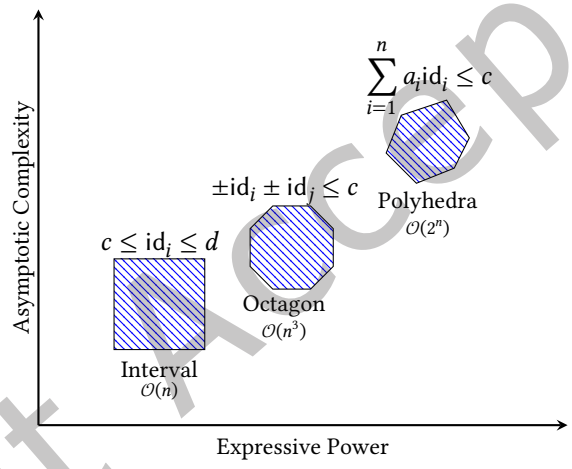


Fig. 3. Comparative illustration of numerical abstract domains of Interval, Octagon, and Polyhedra, where n represent the number of variables.

4.2 Defining Abstract Semantics of Solidity

This section first recalls the basics of three widely used abstract domains, namely Intervals, Octagons, and Polyhedra [11, 13, 32]. For each domain, we describe the corresponding Galois connection between the concrete and abstract semantics, which establishes the soundness of the corresponding abstraction. We also present an illustrative example demonstrating how the same concrete semantics is represented in each abstract domain.

4.2.1 Domain of Intervals. Following the foundational framework established by Cousot and Cousot [11], we utilize the interval domain to approximate our Solidity semantics. The key idea is to replace exact concrete values with intervals that represent the range of values relevant to the analysis. For example, if a variable x may take any value from the concrete set $\{1, 2, 3, 4, 5\}$, the interval abstraction represents this set by the interval $[1, 5]$. Although the interval is less precise than the exact set of values, it safely over-approximates all possible concrete

Just Accepted

values of x. Abstract interpretation similarly replaces concrete operations with sound abstract counterparts. For instance, the concrete computation $3 + 5 = 8$ is represented in the abstract domain as $[3,3] \,\overline{+}\, [5,5] = [8,8]$. This way, computations performed in the abstract domain safely approximate all corresponding concrete computations while remaining computationally tractable. Below, we define the abstraction and concretization functions that map concrete numerical values to the interval domain and vice-versa, and the Galois Connection between them.

Let $\mathcal{L}_c = \langle \wp(\mathbb{R}), \subseteq, \emptyset, \mathbb{R}, \cap, \cup \rangle$ denote the concrete lattice, where $\wp(\mathbb{R})$ is the powerset of the numerical values $\mathbb{R}$, ordered by set inclusion. The abstract domain of intervals is defined as $\mathbb{I} = \{[l,h] \mid l \in \mathbb{R} \cup \{-\infty\},\ h \in \mathbb{R} \cup \{+\infty\},\ l \leq h\} \cup \{\bot\}$, forming the abstract lattice $\mathcal{L}_a = \langle \mathbb{I}, \sqsubseteq, \bot, [-\infty, +\infty], \sqcap, \sqcup \rangle$, where (i) $[l_1,h_1] \sqsubseteq [l_2,h_2] \iff l_2 \leq l_1 \wedge h_1 \leq h_2$, (ii) $[l_1,h_1] \sqcap [l_2,h_2] = [\max(l_1,l_2),\ \min(h_1,h_2)]$, and (iii) $[l_1,h_1] \sqcup [l_2,h_2] = [\min(l_1,l_2),\ \max(h_1,h_2)]$.

The abstraction function $\alpha_{\mathbb{I}} : \wp(\mathbb{R}) \to \mathbb{I}$ and the concretization function $\gamma_{\mathbb{I}} : \mathbb{I} \to \wp(\mathbb{R})$, forming the Galois connection $\langle \mathcal{L}_c, \alpha_{\mathbb{I}}, \gamma_{\mathbb{I}}, \mathcal{L}_a \rangle$, are defined as follows. $\forall S \in \wp(\mathbb{R})$ and $\forall \overline{x} \in \overline{\mathbb{I}}$:

$$\alpha_{\mathbb{I}}(S) = \begin{cases} \bot & \text{if } S = \emptyset \\ [l,h] & \text{if } min(S) = l \wedge max(S) = h \\ [-\infty,h] & \text{if } \nexists min(S) \wedge max(S) = h \\ [l,+\infty] & \text{if } min(S) = l \wedge \nexists max(S) \\ [-\infty,+\infty] & \text{if } \nexists min(S) \wedge \nexists max(S); \end{cases} \qquad \gamma_{\mathbb{I}}(\overline{x}) = \begin{cases} \emptyset & \text{if } \overline{x} = \bot \\ \{k \in \mathbb{R} \mid l \leq k \leq h\} & \text{if } \overline{x} = [l,\ h] \\ \{k \in \mathbb{R} \mid k \leq h\} & \text{if } \overline{x} = [-\infty,\ h] \\ \{k \in \mathbb{R} \mid l \leq k\} & \text{if } \overline{x} = [l,\ +\infty] \\ \mathbb{R} & \text{if } \overline{x} = [-\infty,\ +\infty]. \end{cases}$$

The sound abstract semantics for expressions, $\overline{\mathscr{S}} : (E \cup B) \to (\overline{\Sigma} \to \mathbb{I} \cup \{true, false, \top_B\})$, where $\top_B$ denotes "*may be true or may be false*" over $\mathbb{I}$ is defined as:

$$\overline{\mathscr{S}}[\![v]\!] = \{(\overline{\sigma}, [v,v]) \mid \overline{\sigma} \in \overline{\Sigma}\}, \quad \overline{\mathscr{S}}[\![\mathsf{id}]\!] = \{(\overline{\sigma}, \overline{\sigma}(\mathsf{id})) \mid \overline{\sigma} \in \overline{\Sigma}\}$$
$$\overline{\mathscr{S}}[\![\exp_1 \,\overline{\oplus}\, \exp_2]\!] = \{(\overline{\sigma}, \overline{v}_1 \,\overline{\oplus}\, \overline{v}_2) \mid (\overline{\sigma}, \overline{v}_1) \in \overline{\mathscr{S}}[\![\exp_1]\!], (\overline{\sigma}, \overline{v}_2) \in \overline{\mathscr{S}}[\![\exp_2]\!]\}$$
$$\overline{\mathscr{S}}[\![\exp_1 \,\overline{\oslash}\, \exp_2]\!] = \{(\overline{\sigma}, \overline{v}_1 \,\overline{\oslash}\, \overline{v}_2) \mid (\overline{\sigma}, \overline{v}_1) \in \overline{\mathscr{S}}[\![\exp_1]\!], (\overline{\sigma}, \overline{v}_2) \in \overline{\mathscr{S}}[\![\exp_2]\!]\}$$

For example, the abstract counter-part of the arithmetic operators $\overline{\oplus} \in \{+,-,\times\}$ and relational operators $\overline{\geq} \in \overline{\oslash} = \{\geqslant, \leqslant, <, >, ==\}$ over $\mathbb{I}$ are defined below to over-approximate the concrete result sets. Similarly, other relational operators are defined over $\mathbb{I}$.

$$[l_1,h_1] \,\overline{\oplus}\, [l_2,h_2] = \begin{cases} [\,l_1 + l_2,\ h_1 + h_2\,] & \text{if } \overline{\oplus} \text{ is } \overline{+} \\ [\,l_1 - h_2,\ h_1 - l_2\,] & \text{if } \overline{\oplus} \text{ is } = \\ [\,\min P,\ \max P\,] & \text{if } \overline{\oplus} \text{ is } \overline{\times} \\ \text{where } P = \{l_1 \times l_2, l_1 \times h_2, h_1 \times l_2, h_1 \times h_2\}. \end{cases} \qquad [l_1,h_1] \,\overline{\geq}\, [l_2,h_2] = \begin{cases} \text{true} & \text{if } l_1 \geq h_2 \\ \text{false} & \text{if } h_1 < l_2 \\ \top & \text{otherwise} \end{cases}$$

4.2.2 Relational Abstract Domain of Octagons. We employ the weakly relational octagon domain, formally defined by Antoine Miné [32], to improve analysis precision over intervals by capturing relationships between variables. This domain defines invariants as a system of constraints of the form: $\pm \mathsf{id}_i \pm \mathsf{id}_j \leq v$, where $\mathsf{id}_i, \mathsf{id}_j$ represent program variables and $v \in \mathbb{R}$.

Encoding octagonal constraints. Given a set of constraints of the form $\mathsf{id}_j - \mathsf{id}_i \leq v$, we can represent them using Difference Bound Matrix (DBM) $\mathbf{m} = [\mathsf{m}_{ij}]$ of size $n \times n$, as follows.

$$\mathsf{m}_{ij} \triangleq \begin{cases} v & \text{if } (\mathsf{id}_j - \mathsf{id}_i \leq v), \quad \mathsf{id}_i, \mathsf{id}_j \in \mathtt{Var} \text{ and } v \in \mathbb{R}, \\ \infty & \text{otherwise.} \end{cases}$$

Just Accepted

To encode general octagonal constraints $\pm \mathsf{id_i} \pm \mathsf{id_j} \leq v$, a DBM $\mathbf{m}$ of dimension $n \times n$ over $\mathtt{Var}$ is extended to a coherent DBM (CDBM) m' of size $2n \times 2n$, over the enhanced set $\mathtt{Var}' = \{\mathsf{id}'_1, \dots, \mathsf{id}'_{2n}\}$, where id'_{2i-1} refers to $\mathsf{id_i}^+$ and id'_{2i} refers to $\mathsf{id_i}^-$.

Galois Connection between concrete powerset of numerical domain and abstract octagonal domain. Let $\mathbb{O}$ be the set of all CDBMs comprising the octagon domain. The abstract domain is $\mathcal{L}_a = \langle \mathbb{O}_\bot, \sqsubseteq, \mathbf{m}_\bot, \mathbf{m}_\top, \sqcap, \sqcup \rangle$, where $\mathbb{O}_\bot = \mathbb{O} \cup \{\mathbf{m}_\bot\}$, with $\mathbf{m}_\top$ denoting the top element and $\mathbf{m}_\bot$ the bottom element. The ordering relation, meet, and join operations on $\mathcal{L}_a$ are defined as follows: (i) $\forall \mathbf{m}, \mathbf{n} \in \mathbb{O}_\bot : \mathbf{m} \sqsubseteq \mathbf{n} \iff \forall i, j \ : \ \mathbf{m}_{ij} \leqslant \mathbf{n}_{ij}$, (ii) $\forall \mathbf{m}, \mathbf{n} \in \mathbb{O}_\bot : (\mathbf{m} \sqcap \mathbf{n}) = \mathbf{m}'$ where $\forall i, j \ : \ \mathbf{m}'_{ij} = \min(\mathbf{m}_{ij}, \mathbf{n}_{ij})$, and (iii) $\forall \mathbf{m}, \mathbf{n} \in \mathbb{O}_\bot : (\mathbf{m} \sqcup \mathbf{n}) = \mathbf{m}'$ where $\forall i, j \ : \ \mathbf{m}'_{ij} = \max(\mathbf{m}_{ij}, \mathbf{n}_{ij})$. The abstraction function $\alpha_{\mathbb{O}}$ and the concretization function $\gamma_{\mathbb{O}}$, forming Galois connection $\langle \mathcal{L}_c, \ \alpha_{\mathbb{O}}, \ \gamma_{\mathbb{O}}, \ \mathcal{L}_a \rangle$, are defined as follows. $\forall S \in \wp(\mathbb{R}^n)$ and $\forall \mathbf{m} \in \mathbb{O}_\bot$:

- if $S = \emptyset$: $\quad \alpha_{\mathbb{O}}(S) \triangleq \mathbf{m}_\bot$.
- if $S \neq \emptyset$: $\quad \alpha_{\mathbb{O}}(S) = \mathbf{m}$ where, for all i, j,

$$\mathbf{m}_{ij} \triangleq \begin{cases} \max\{\sigma(\mathsf{id}_l) - \sigma(\mathsf{id}_k) \mid \sigma \in S\}, & \text{if } i = 2k-1, j = 2l-1 \text{ or } i = 2l, j = 2k, \\ \max\{\sigma(\mathsf{id}_l) + \sigma(\mathsf{id}_k) \mid \sigma \in S\}, & \text{if } i = 2k, j = 2l-1, \\ \max\{-\sigma(\mathsf{id}_l) - \sigma(\mathsf{id}_k) \mid \sigma \in S\}, & \text{if } i = 2k-1, j = 2l. \end{cases}$$

$$\gamma_{\mathbb{O}}(\mathbf{m}) = \begin{cases} \emptyset & \text{if } \mathbf{m} = \mathbf{m}_\bot, \\ \mathbb{R}^n & \text{if } \mathbf{m} = \mathbf{m}_\top, \\ \{(k_1, \dots, k_n) \in \mathbb{R}^n \mid (k_1, -k_1, \dots, k_n, -k_n) \\ \in \mathrm{dom}(\mathbf{m}) \text{ and } \forall i, j \ : \ \mathsf{id}_j - \mathsf{id}_i \leq \mathbf{m}_{ij}\} & \text{otherwise.} \end{cases}$$

4.2.3 Relational Abstract Domain of Polyhedra. The polyhedra domain, as established in the seminal work of Cousot and Halbwachs [13], provides a mechanism to model arbitrary linear relationships between program variables. Within this theoretical framework, a polyhedron is formalized as a region in the n-dimensional space $\mathbb{R}^n$ bounded by a finite set of hyperplanes. Consider the set of Solidity variables $\mathtt{Var} = \{ id_1, id_2, \dots, id_n \}$. We represent $\vec{c} = \langle c_1, c_2, \dots, c_n \rangle \in \mathbb{R}^n$ for an n-tuple of real coefficients, and $\vec{\mathsf{id}} = \langle \mathsf{id}_1, \mathsf{id}_2, \dots, \mathsf{id}_n \rangle$ for the corresponding vector of variables. According to the standard formulation, a linear inequality over $\mathbb{R}^n$ is given by $\beta = \vec{c} \cdot \vec{\mathsf{id}} \geq k$, where $\vec{c} \neq \vec{0}$ and $k \in \mathbb{R}$. Each such inequality defines an affine half-space in $\mathbb{R}^n$. If a set $\Theta = \{\beta_1, \beta_2, \dots, \beta_m\}$ of linear inequalities constrains $\mathbb{R}^n$, then the intersection of their corresponding half-spaces yields a convex polyhedron $\mathbb{P} = (\Theta, n)$. Equivalently, a convex polyhedron can also be described by its frame representation, that is, as the collection of its generators (vertices and rays) [9]. Conversely, any set of solutions to a system of linear inequalities Θ defines a polyhedron $\mathbb{P} = (\Theta, n)$ in $\mathbb{R}^n$.

Galois Connection between concrete powerset of numerical domain and abstract polyhedra domain. Let $\mathcal{L}_c = \langle \wp(\mathbb{R}^n), \subseteq, \emptyset, \mathbb{R}^n, \cap, \cup \rangle$ be the concrete lattice over $\mathbb{R}^n$. The set of convex polyhedra $\mathbb{P}$ with partial order $\sqsubseteq$ forms an abstract lattice $\mathcal{L}_a = \langle \mathbb{P}, \sqsubseteq, \mathbb{P}_\bot, \mathbb{P}_\top, \sqcap, \sqcup \rangle$. For $\mathsf{P}_1, \mathsf{P}_2 \in \mathbb{P}$:

- $\mathsf{P}_1 \sqsubseteq \mathsf{P}_2$ iff $\gamma(\mathsf{P}_1) \subseteq \gamma(\mathsf{P}_2)$, where $\gamma(\mathsf{P})$ denotes its concrete solutions.
- $\mathsf{P}_1 \sqcap \mathsf{P}_2$ is the convex polyhedron such that $\gamma(\mathsf{P}_1 \sqcap \mathsf{P}_2) = \gamma(\mathsf{P}_1) \cap \gamma(\mathsf{P}_2)$.
- $\mathsf{P}_1 \sqcup \mathsf{P}_2$ is not guaranteed to be convex, so the smallest convex polyhedron containing the union is computed as their convex hull.

Just Accepted

```
function withdraw(uint256 fee) external payable{
    uint256 balance = msg.value;
    require(fee < balance);
    uint256 net = balance - fee;
    uint256 refund = fee / 2;
    assert(net >= 0);
}
```

Fig. 4. Solidity code snippet of the withdraw function used to illustrate the precision hierarchy across abstract domains.

An environment $\sigma \in \Sigma \triangleq \mathtt{Var} \mapsto \mathbb{R}$ assigns each variable $\mathsf{id} \in \mathtt{Var}$ to its value in $\mathbb{R}$. The concretization function $\gamma_{\mathbb{P}} : \mathbb{P} \to \wp(\Sigma)$ is defined as:

$$\gamma_{\mathbb{P}}(\mathsf{P}) = \begin{cases} \emptyset & \text{if } \mathsf{P} = \mathsf{P}_{\bot}, \\ \mathbb{R}^n & \text{if } \mathsf{P} = \mathsf{P}_{\top}, \\ \{\sigma \in \Sigma \mid \forall (\vec{c} \cdot \vec{\mathsf{id}} \geq v) \in \Theta : \vec{c} \cdot \sigma(\vec{\mathsf{id}}) \geq v\} & \text{otherwise.} \end{cases}$$

Notably, an abstraction function is not always available in the polyhedra domain, as some sets of vectors cannot be tightly over-approximated by a convex closed polyhedron [13]. In such cases, $\alpha_{\mathbb{P}}(\mathsf{S})$ denotes a (possibly minimal) polyhedron in $\mathbb{P}$ with $\gamma_{\mathbb{P}}(\alpha_{\mathbb{P}}(\mathsf{S})) \supseteq \mathsf{S}$.

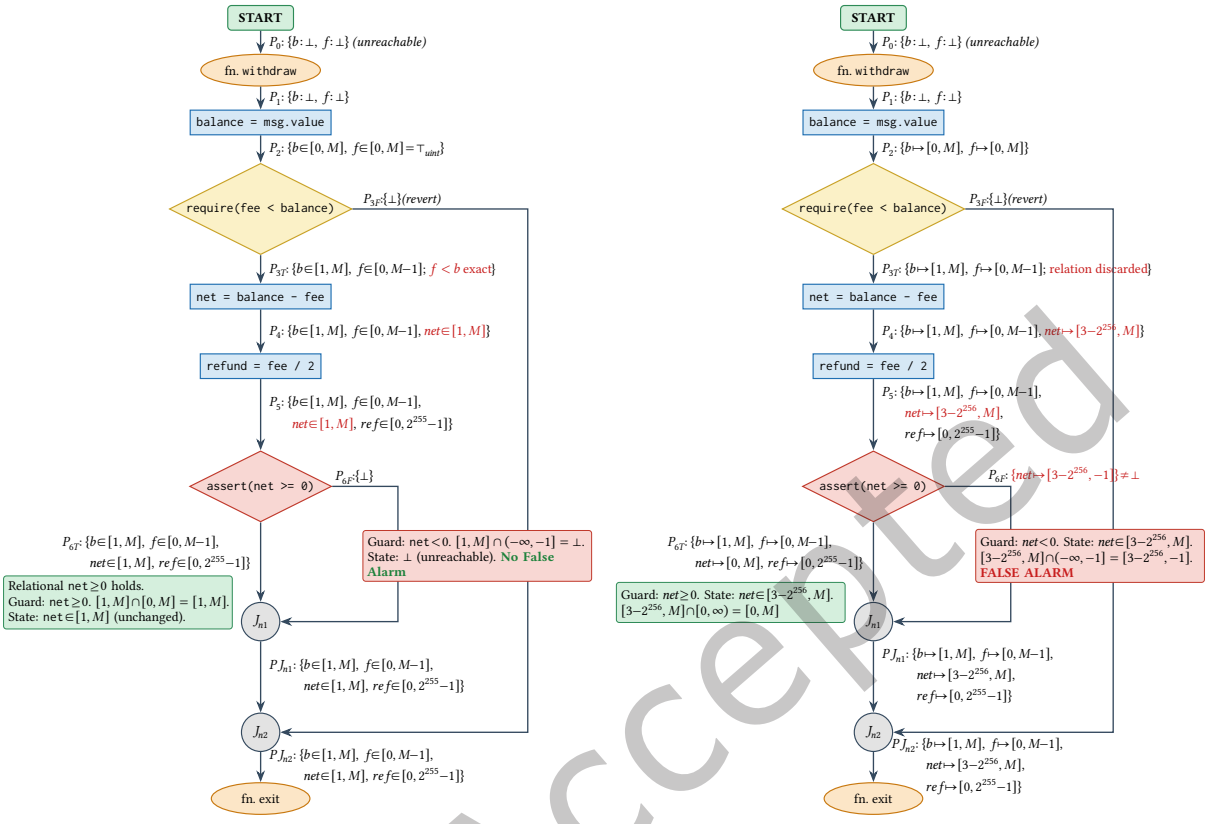
Example 4.2. Figure 4 shows the solidity code snippet of the withdraw function used to compare the precision of different abstract domains. The function takes fee as an input parameter and assigns balance from msg.value. Since both values depend on the caller, their exact values are not known during analysis. The condition require(fee < balance) restricts the possible values of fee and balance. The function then computes net = balance - fee and refund = fee / 2, followed by the assertion assert(net >= 0). This example highlights how different abstract domains handle the information provided by the require condition and how this affects the precision of the analysis. For simplicity, we use b for balance, f for fee, *net* for net, *ref* for refund, and $M = 2^{256} - 1$ to denote the maximum value representable by the `uint256` type.

Figures 5 and 6 show all four domains over the same control-flow graph of Figure 4. Sub-figure 5b covers the analysis over the non-relational abstract domains, whereas Sub-figures 6a–6b cover the same over relational abstract domains. Each program point is annotated with the abstract state computed by the respective domain.

Key Observations on Analysis Precision Across Abstract Domains.

- In the concrete domain (Figure 5a), the guard require (fee < balance) keeps the exact fact $f < b$ at every later program point. So net = balance - fee is always at least 1, the check net >= 0 always passes, and no alarm is raised. This is the ground truth that every abstract domain should try to match.
- The interval domain (Figure 5b) stores only a separate range for each variable and keeps no relation between them. After the guard at P_3^T it knows $b \mapsto [1, M]$ and $f \mapsto [0, M-1]$, but it no longer knows that f is smaller than b. For the subtraction at P_4, the interval domain applies the rule from Section 4.2, $[l_1, h_1] - [l_2, h_2] = [\, l_1 - h_2,\ h_1 - l_2 \,]$. Substituting the bounds of b and f gives *net* $= [\, 1-(M-1),\ M-0 \,] = [\, 3-2^{256},\ M \,]$. Because the two variables are treated independently, this takes the impossible case $b = 1$ together with $f = M - 1$, even though the guard rules it out. This range includes negative values, so the interval domain cannot prove net >= 0 and reports a false alarm. The cause is not the subtraction rule itself but the loss of the relation $f < b$ before the subtraction is computed.
- The octagon domain (Figure 6a) can remember a relationship between two variables, written as $\pm \mathsf{id}_i \pm \mathsf{id}_j \leq v$. Here it remembers the guard as $f - b \leq -1$, that is, *fee* is smaller than *balance*. It keeps this fact through the subtraction and derives *net* ≥ 1, so the check net >= 0 passes and the false alarm is gone.

Just Accepted



(a) **Concrete domain.** (b) **Interval domain.**

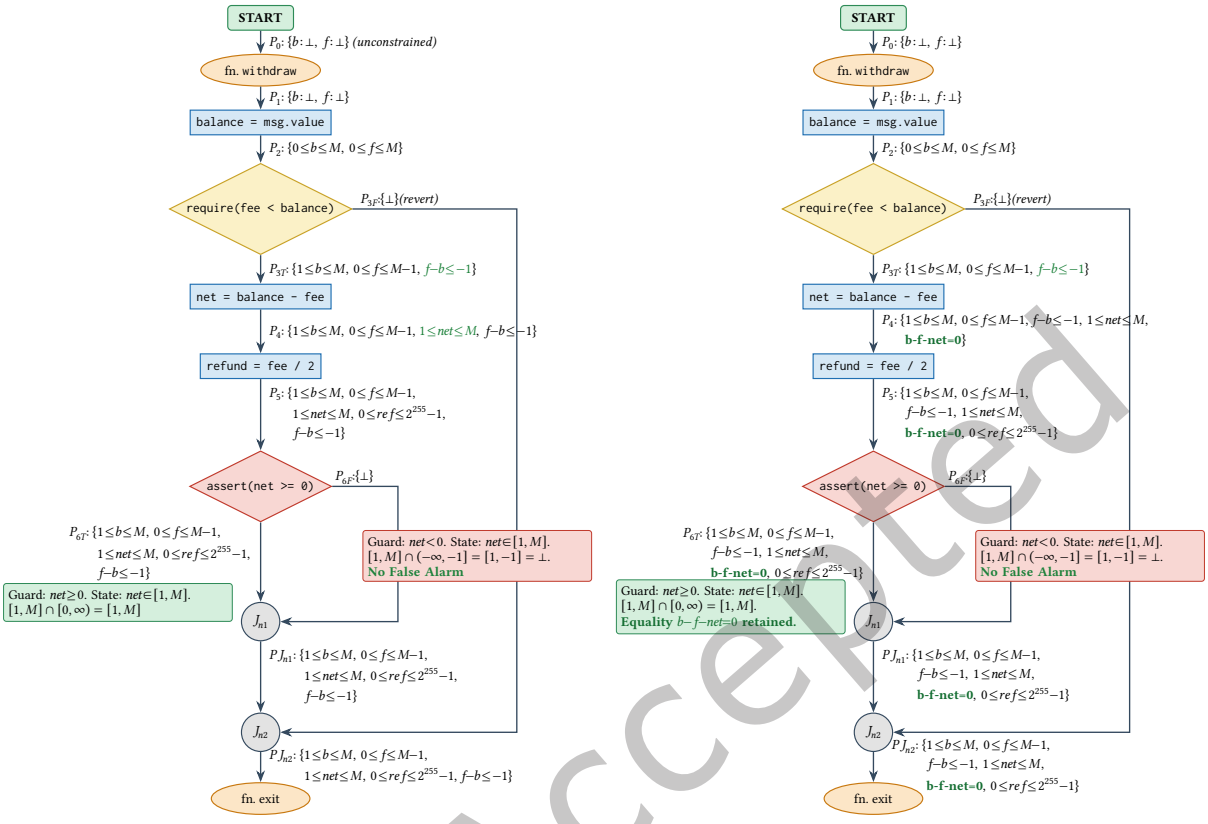
Fig. 5. Analysis of the withdraw function in the non-relational domains: concrete (a) and interval (b).

- The polyhedra domain (Figure 6b) gives the same correct answer as the octagon domain, with no false alarm. The only difference is that it can also remember a link among three variables at once. Besides keeping $f - b \leq -1$, it records the exact equality $b - f - net = 0$, which the octagon domain cannot write because it only handles two variables at a time.
- For this contract the extra relation kept by the polyhedra domain does not change the result, since the octagon constraint is already enough to prove the assertion. The benefit of polyhedra would appear in contracts where the safety of a statement depends on a relation among three or more variables, which the octagon domain cannot represent.
- Overall, the three domains form a precision ordering Interval ⊊ Octagon ⊊ Polyhedra: the interval domain is the least precise and raises a false alarm here; the octagon domain is precise enough to remove it; and the polyhedra domain is the most precise, keeping the full linear relation among balance, fee, and net even when it is not needed for this particular check.

5 Vulnerability Detection using Abstract Semantics

In this section, we describe our proposed SCVD strategies based on the computed abstract semantics over augmented CFG.

Just Accepted



(a) **Octagon domain** (b) **Polyhedra domain.**

Fig. 6. Analysis of the withdraw function in the relational domains: octagon (a) and polyhedra (b).

5.1 Reentrancy Vulnerability via Abstract Semantic Interpretation

Reentrancy vulnerabilities arise when a contract exposes its internal state to external contracts before completing a prior execution context. Exploiting this, an adversary can recursively re-enter the contract, often via a fallback function, and manipulate its control or state in unintended ways. Our framework addresses reentrancy by combining augmented control flow with flow-sensitive abstract interpretation, allowing precise semantic detection of reentrant behaviours.

Modeling Reentrant Paths via Augmented Control Flow. We construct an augmented control flow graph (CFG) by adding synthetic edges that illustrate fallback re-entry paths for reentrant execution. These edges stem from external interaction sites, such as calls, delegate calls, and transfers, looping back to the function entry point that may be triggered through fallback in untrusted external contexts. We also include augmented nodes representing assignment statements that assigns attacker's balance and contract's balance to two ghost variables, simulating a blockchain environment with various account configurations and cryptocurrency balances across multiple accounts. This augmented graph helps the abstract interpreter to analyze both standard and adversarial execution paths, preserving semantic interactions that might be missed in syntactic analysis. This way, modeling each external call using synthetic edges as a re-entry to the contract function, we over-approximate the set of reachable reentrant paths and it is therefore sound for vulnerability detection. Observe that these synthetic edges are

Just Accepted

inserted automatically by the analyzer in a single pass. Specifically, the analyzer traverses the original CFG, identifies nodes corresponding to contract statements that perform external calls, and introduces re-entry edges from those nodes to the appropriate function entry points.

Definition 5.1 (Augmented Control Flow Graph). Let sc be a solidity smart contract. The *augmented control flow graph* of sc is a directed graph $\mathcal{G} = (N, E)$, where:

- N denotes the set of nodes, expressed as $N = N_{\text{stmt}} \cup N_{\text{jn}} \cup \{\text{start}, \text{end}\} \cup N_{\text{b}}$, where N_{stmt} represents the set of nodes corresponding to statements stmt ∈ sc, N_{jn} signifies the set of join nodes that indicate control merge points from various branches, start and end are unique nodes that denote the entry and exit points of the program, and $N_{\text{b}} = \{n_b^c, n_b^a\}$ denotes augmented nodes corresponding to the initial balances in contract's and attacker's accounts, respectively.
- E denotes the set of directed edges, expressed as $E = E_{\text{control}} \cup E_{\text{ext}}$, where $E_{\text{control}} : N \mapsto N$ and $E_{\text{ext}} : N_{\text{ext}} \mapsto N_{\text{jn}}$. Here $N_{\text{ext}} \in N_{\text{stmt}}$ refers to the node associated with the external call sites (e.g., `call()`, `delegatecall()`).

Fixpoint Semantics and Invariant Propagation. The key objective of this phase is to compute abstract collecting semantics of smart contracts, approximating all possible dynamic behavioral properties, without actually executing them. To this aim, we adapt data-flow analysis based fix-point semantic computation in an appropriate abstract domain of interest using the state transition functions $\overline{\mathscr{S}}$ defined earlier.

Given a set of abstract states, $\overline{\Sigma}$ and a set of program points *Label*, the context vector is defined by $\overline{CV} : \ Label \rightarrow \overline{Cx}$, where $\overline{Cx} = \overline{\Sigma}$ represents the collection of contexts. A context vector associated with a program P of size l is, thus, denoted by $\overline{CV}_P = \langle \overline{Cx}_1, \overline{Cx}_2, \ldots, \overline{Cx}_l \rangle$, where $\overline{Cx}_i$ is the context associated with program point corresponding to stmt_i. Let $\overline{DF} : \ \overline{CV} \rightarrow \overline{Cx}$ be a collection of monotone functions representing dataflow computation. For the program P, we therefore have

$$\overline{Cx}_1 = \overline{DF}_1\left(\overline{Cx}_1, \ldots, \overline{Cx}_l\right), \ \overline{Cx}_2 = \overline{DF}_2\left(\overline{Cx}_1, \ldots, \overline{Cx}_l\right), \ \ldots\ldots, \ Cx_n = \overline{DF}_l\left(\overline{Cx}_1, \ldots, \overline{Cx}_l\right)$$

Combining the above functions, we get $\overline{CS} : \ \overline{CV} \rightarrow \overline{CV}$. That is,

$$\overline{CS}\left(\overline{Cx}_1, \ldots, \overline{Cx}_l\right) = \left(\overline{DF}_1\left(\overline{Cx}_1, \ldots, \overline{Cx}_l\right), \ldots, \overline{DF}_l\left(\overline{Cx}_1, \ldots, \overline{Cx}_l\right)\right)$$

Each function $\overline{DF}_i$ includes the transition function defined as follows:

$$\overline{Cx}_i = \bigsqcup_{\text{stmt}_j \in \text{pred}(\text{stmt}_i), \bar{\sigma}_j \in \overline{Cx}_j} \overline{\mathscr{S}}[\![\text{stmt}_i]\!]\left(\overline{\sigma}_j\right) \tag{1}$$

where, $\text{pred}(\text{stmt}_i)$ is the set of predecessors of the statement stmt_i. Starting from the initial context vector $\overline{CV}_P = \langle \bot, \ldots, \bot \rangle$, which is the bottom element of the abstract lattice $\mathcal{L}_a = (\overline{\Sigma}, \sqsubseteq, \sqcap, \sqcup, \top, \bot)$ representing the abstract domain, the computation of the least fixpoint of $\overline{CS}$ results in the abstract collecting semantics for P. The overall algorithmic steps to compute collecting semantics of smart contracts in an abstract domain of interest are depicted in Algorithm 1. Observe that, once the new collecting semantics $\overline{Cx_i}'$ is computed, a subsumption check is performed (Step 10) to determine whether it is strictly more informative than the existing semantics $\overline{Cx_i}$. If so, the widening operator is applied (Step 11) to accelerate convergence and ensure termination in the presence of possibly infinite ascending chains. We specifically adopt the widening strategies analyzed by Cortesi and Zanioli [10], implementing the threshold-based widening operator for the interval domain as defined below.

Definition 5.2 (Widening for intervals). Consider a lattice of intervals $\mathbb{I}$. Let k be a fixed positive integer constant, and I be any set of indices. The following defines a threshold widening operator defined on $\mathbb{I}$ for any finite set

Just Accepted

Algorithm 1 Fixpoint-Based Collecting Semantics Computation

Input: Augmented CFG $\mathcal{G} = (N, E)$, abstract domain $\mathcal{D}$ (Interval, Octagon, Polyhedra)
Output: Collecting semantics mapping $\overline{CS}$
Initialize $\overline{Cx_i} \leftarrow \bot$ for all $n_i \in N$
repeat
  for all $n_i \in N$ **do**
    $\overline{Cx_i}' \leftarrow \bot$
    for all $n_j \in \text{pred}(n_i)$ **do**
      $\overline{Cx_i}' \leftarrow \overline{Cx_i}' \sqcup \left(\bigsqcup_{\overline{\sigma} \in \overline{Cx_j}} \overline{\mathscr{F}}[\![s_i]\!](\overline{\sigma}) \right)$
    end for
    if $\overline{Cx_i}' \not\sqsubseteq \overline{Cx_i}$ **then**
      $\overline{Cx_i} \leftarrow \text{Widen}(\overline{Cx_i}, \overline{Cx_i}')$
    end if
  end for
until no $\overline{Cx_i}$ changes for any i
return $\langle \overline{Cx_1}, \ldots, \overline{Cx_l} \rangle$

Algorithm 2 Semantic Detection of Reentrancy Vulnerability

Input: Augmented CFG $\mathcal{G} = (N, E)$, abstract domain $\mathcal{D}$
Output: Vulnerability report $\mathcal{R}$
$\overline{\mathcal{CS}} \leftarrow \text{FixpointCollectingSemantics}(\mathcal{G}, \mathcal{D})$ // Invoke Algorithm 1
Initialize $\mathcal{R} \leftarrow \emptyset$
  $\mathcal{V} \leftarrow \{\texttt{att}, \texttt{con}, \texttt{credit}[a]\}$
for all reentrant execution paths $p \in \mathcal{G}$ **do**
  $\overline{\mathcal{CS}}_{\text{init}} \leftarrow \text{GetInitContext}(\overline{\mathcal{CS}}, p, \mathcal{V})$
  $\overline{\mathcal{CS}}_{\text{final}} \leftarrow \text{GetFinalContext}(\overline{\mathcal{CS}}, p, \mathcal{V})$
  for all $v \in \mathcal{V}$ **do**
    if $\overline{\mathcal{CS}}_{\text{init}}(v) \neq \bot$ and $\overline{\mathcal{CS}}_{\text{final}}(v) \neq \bot$ **then**
      $\Delta_v \leftarrow \overline{\mathcal{CS}}_{\text{final}}(v) \ominus \overline{\mathcal{CS}}_{\text{init}}(v)$
    end if
  end for
  if $\neg(\Delta_{\texttt{att}} = \Delta_{\texttt{con}} = \Delta_{\texttt{credit}[a]})$ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\text{Reentrancy at } p\}$
  end if
end for
return $\mathcal{R}$

A by $\nabla^k(\{\bot\}) = \bot, \nabla^k(\{\bot\} \cup A) = \nabla^k(A), \nabla^k(\{[l_i, h_i] \ : \ i \in I\}) = [u_1, u_2]$, where $u_1 = min\{l_i \ : \ i \in I\}$ if $min\{l_i \ : \ i \in I\} > -k$ else $-\infty, u_2 = max\{h_i \ : \ i \in I\}$ if $max\{h_i \ : \ i \in I\} < k$ else $+\infty$.

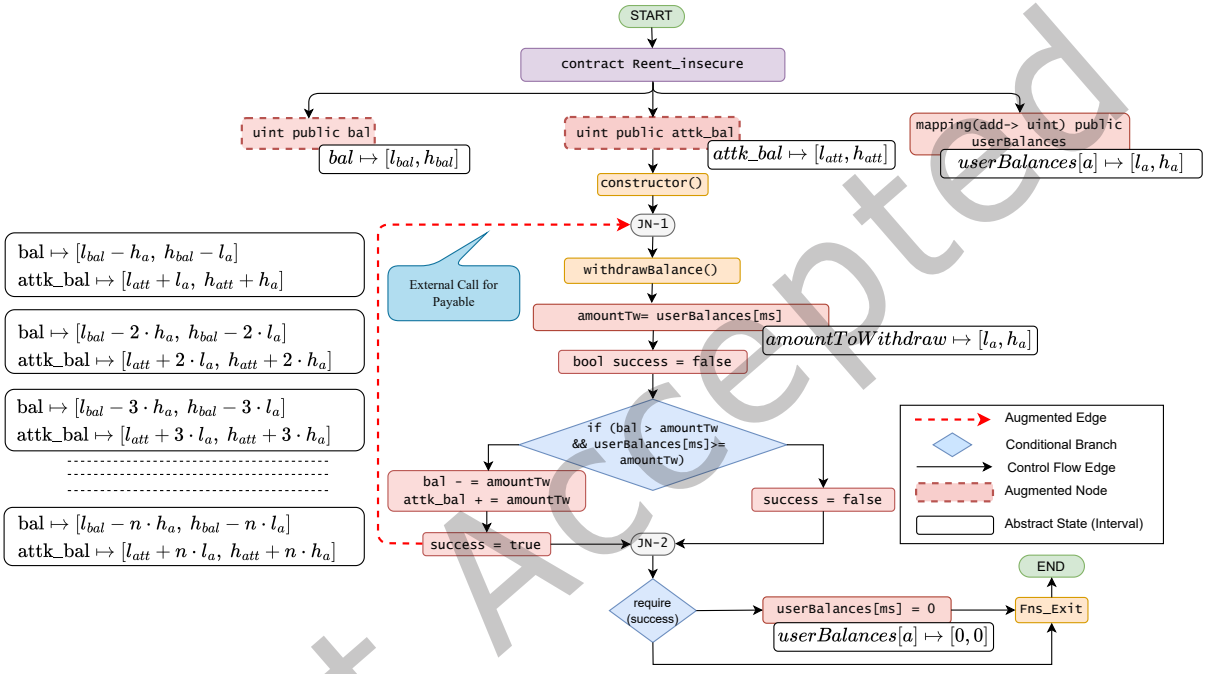
At convergence, the resulting mapping $\langle \overline{Cx}_1, \ldots, \overline{Cx}_l \rangle$ captures the least fixpoint solution over the abstract domain $\mathcal{D}$ and provides a sound over-approximation of the collecting semantics for the program. These invariants serve as the semantic foundation for statically detecting vulnerabilities such as reentrancy, integer overflow and underflow, timestamp dependency, and TOD, without requiring program execution.

Semantic Characterization of Reentrancy Vulnerability. To semantically detect reentrancy vulnerabilities, we analyze the evolution of abstract states along reentrant execution paths derived from the collecting semantics in Algorithm 1. Our method observes changes in key program variables across function invocations, particularly focusing on attacker-held balance (`att`), contract balance (`con`), and user specific credit mappings (`credit[a]`) where a denotes the address accessing the credit mapping. The precise runtime identity of this address is intentionally abstracted away. Following the standard principle of Abstract Interpretation, dynamic values are replaced by sound over-approximations whenever their exact runtime values are unavailable or unnecessary for the property under verification. Since reentrancy may be triggered by any external caller, we conservatively model a as an arbitrary address, including a potentially malicious caller. A reentrant path $p \in \mathcal{G}$ is a sequence $p = \langle n_{\text{entry}}, \ldots, n_{\text{ext}}, n_{\text{entry}}, \ldots, n_{\text{exit}} \rangle$ where $n_{\text{ext}} \in N_{\text{ext}}$ denotes an external call, and the fallback edge $(n_{\text{ext}}, n_{\text{entry}}) \in E_{\text{ext}}$ models reentrancy by redirecting control back to the function entry. Let $\mathcal{V} = \{\texttt{att}, \texttt{con}, \texttt{credit}[a]\}$ be the variables of interest. Our objective is to record the changes of their abstract values along each reentrant path p during the fixpoint iteration (Step 4-13) in Algorithm 1. Let $\overline{\mathcal{CS}}_{\text{init}}$ and $\overline{\mathcal{CS}}_{\text{final}}$ represent the semantics context where ($\forall v \in \mathcal{V}$: $\overline{\mathcal{CS}}_{\text{init}}(v) \neq \bot$ and $\overline{\mathcal{CS}}_{\text{final}}(v) \neq \bot$) captured during initial and final iterations. We compute their difference as an abstract delta: $\Delta_v = \overline{\mathcal{CS}}_{\text{final}}(v) \ominus \overline{\mathcal{CS}}_{\text{init}}(v)$ for all $v \in \mathcal{V}$. Here, $\ominus$ denotes the domain-specific abstract difference operator associated with the chosen abstract domain $\mathcal{D}$, such as interval subtraction, relational difference in the octagon domain, or geometric difference in the polyhedral domain. We maintain an invariant that signifies the preservation of semantic balance along any reentrant path: $\Delta_{\texttt{att}} = \Delta_{\texttt{con}} = \Delta_{\texttt{credit}[a]}$. This invariant ensures that any increase in the attacker's balance is countered by a corresponding decrement in the contract's balance and the user's credit; a breach of this invariant indicates a possible reentrancy problem. Because a is modelled as representing any caller, as described above, the balance-preservation invariant is checked in a form that must hold for every address that may invoke the withdrawal function. If the invariant holds for the symbolic

Just Accepted

```
1  contract Reentrancy_insecure {
2    mapping(address => uint) private userBalances;
3    function withdrawBalance() public {
4      uint amountTw = userBalances[msg.sender];
5      // <yes> <report> REENTRANCY
6      (bool success, ) = msg.sender.call.value(amountTw)("");
7      require(success);
8      userBalances[msg.sender] = 0;}
9  }
```

(a) The vulnerable Reentrancy_insecure contract



(b) Illustration of semantics based reentrancy detection using augmented CFG

Fig. 7. The vulnerable Reentrancy_insecure contract (a) and its augmented CFG (b).

address, it holds for all corresponding concrete callers; if it is violated, the analysis reports a potential reentrancy vulnerability. This verification is possible because, before abstract interpretation begins, the contract is reduced to the numerical variables relevant to vulnerability detection, namely the Ether flow among the contract, the caller, and the credit ledger. Algorithm 2 formalizes this detecting process.

Below, we illustrate our approach using two examples, where the former demonstrates how our approach detects a reentrancy vulnerability, while the latter demonstrates how it eliminates a false positive.

Example 5.3. Consider the code snippet shown in Figure 7a. Figure 7b shows its augmented CFG, where we use `bal` and `attk_bal` to model the contract and attacker's balances. Join nodes (JN-1 and JN-2) show where control flows merge, and red dotted fallback edges show synthetic reentrant calls. We detect reentrancy using abstract interpretation over the interval domain, where each entry `userBalances[a]` is modeled as an abstract interval $[l_a, h_a]$. The initial collecting semantics $\overline{\mathcal{CS}}_{\text{init}}$ (computed via Algorithm 1) captures the abstract state after

Just Accepted

initialization: $\overline{\mathcal{CS}}_{\text{init}}(\texttt{bal}) = [l_{bal},\ h_{bal}], \overline{\mathcal{CS}}_{\text{init}}(\texttt{attk_bal}) = [l_{att},\ h_{att}], \overline{\mathcal{CS}}_{\text{init}}(\texttt{userBalances[a]}) = [l_a,\ h_a]$. Note that our analysis tool automatically adds the semantic guard `if bal > amountTw && userBalances[ms] >= amountTw`, where `ms` abbreviates `msg.sender`, to ensure sound exploration of feasible reentrant paths. These semantic guards are not manually inserted into the source code. Instead, they are generated automatically during a preprocessing phase before the abstract fixpoint computation. Specifically, the analyzer introduces ghost variables for the contract balance `bal` and the attacker balance `attk_bal`, and rewrites each recognized external value-transfer statement into a guarded abstract transition. The transfer amount is extracted from the call site, such as `.call{value: amount}`, `.transfer(amount)`, `.send(amount)`, or the legacy `call.value(amount)()` pattern, while `credit` is obtained from the caller's balance mapping. In the contract of Figure 7a, `userBalances[msg.sender]` is reset only after the external call. Hence, when the fallback edge is taken, the re-entered invocation can still read the same positive credit, and the generated guard can remain satisfiable as long as the contract balance is sufficient. Every time a feasible reentrant call occurs, the abstract state changes by $\texttt{bal} -= \texttt{amountTw}, \texttt{attk_bal} += \texttt{amountTw}$, indicating that the attacker's balance increases while the contract's balance decreases cumulatively. After n re-entries, the abstract state becomes $\overline{\mathcal{CS}}_{\text{final}}(\texttt{bal}) = [l_{bal} - n \cdot h_a,\ h_{bal} - n \cdot l_a]$, $\overline{\mathcal{CS}}_{\text{final}}(\texttt{attk_bal}) = [l_{att} + n \cdot l_a,\ h_{att} + n \cdot h_a]$, $\overline{\mathcal{CS}}_{\text{final}}(\texttt{userBalances[a]}) = [0, 0]$. The resulting abstract deltas $\Delta_{\texttt{attk_bal}}, \Delta_{\texttt{bal}}$, and $\Delta_{\texttt{userBalances[a]}}$ are pairwise unequal. This violates the balance preservation invariant defined in Algorithm 2 and confirms the vulnerability.

Example 5.4. Figure 8 presents the `Reentrancy_safe` contract and its augmented CFG. Unlike the vulnerable contract in Example 5.3, this contract resets `rewardsForA[msg.sender]` to zero before the external call, following the checks-effects-interactions order. During the initial invocation of `withdrawReward()`, the caller's recorded reward is assigned to `amountToWithdraw`, and the condition `require(amountToWithdraw > 0)` restricts it to positive values. Before the generated transfer guard is evaluated, the function executes `rewardsForA[msg.sender] = 0`, so the abstract state at the guard records $\texttt{rewardsForA[msg.sender]} \mapsto [0, 0]$. The generated guard is $\texttt{bal} > \texttt{amountToWithdraw} \ \wedge \ \texttt{rewardsForA[msg.sender]} \geq \texttt{amountToWithdraw}$. Since `amountToWithdraw` is positive while the caller's remaining credit is zero, the second conjunct is false. Hence, the true branch of the injected abstract transition, which contains the updates to `bal` and `attk_bal` and its outgoing synthetic re-entry edge, maps to $\bot$. The augmented CFG therefore contains no feasible reentrant path, and the balance-preservation invariant in Algorithm 2 is not violated. This contrasts with Example 5.3, where the credit mapping is reset after the external call and the generated reentrant cycle remains feasible. In our experiments, Mythril and Oyente+ report reentrancy for this safe contract, whereas Slither and our analysis report no reentrancy.

5.2 Integer Overflow Underflow Vulnerability via Abstract Semantic Interpretation

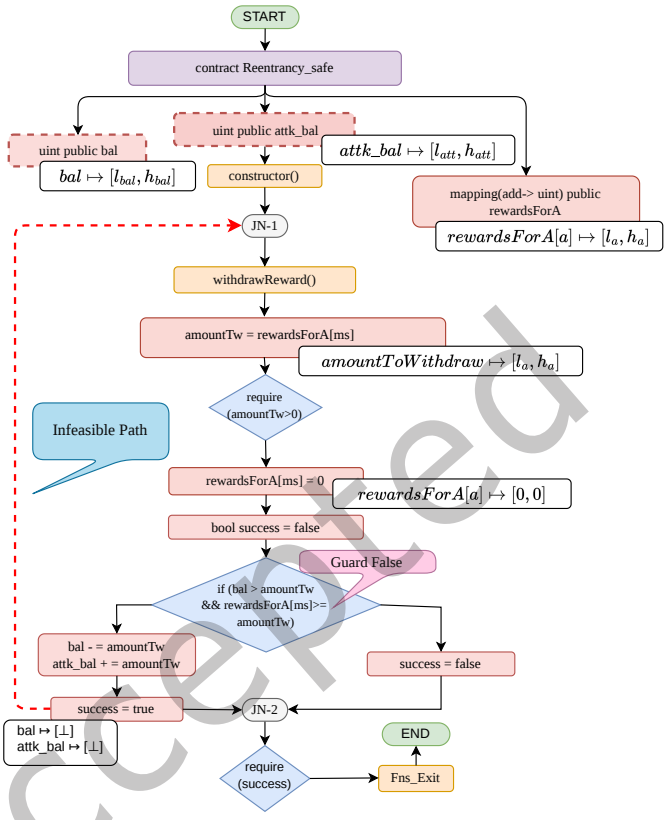
Integer overflow and underflow are critical vulnerabilities that arise when arithmetic operations produce values that exceed the bounds of the underlying integer datatype. In Solidity, integers are represented through fixed-size data types, with permissible range 0 to $(2^X - 1)$ for `uintX`, whereas $-2^{(X-1)}$ to $(2^{(X-1)} - 1)$ for `intX` where `X` varies from 8 to 256 in steps of 8. Any arithmetic operation that yields a value beyond these specified limits results in an overflow or underflow, a vulnerability that can be exploited by malicious actors to illegitimately modify contract states and transfer funds.

The interval abstract domain has been shown to be effective in detecting integer-overflow bugs [26]. The detection of integer overflow and underflow vulnerabilities proceeds via a systematic fixpoint semantics computation over the interval domain, as formalized in Algorithm 3. As defined in Section 4.2, all arithmetic and relational operators ($\overline{\oplus}$ *and* $\overline{\oslash}$) over the interval domain are sound by construction, ensuring correctness of the analysis.

Just Accepted

```
1  pragma solidity ^0.8.20;
2
3  contract Reentrancy_safe {
4
5  mapping(address => uint) private rewardsForA;
6
7  function depositReward(address _beneficiary)
8  public payable {
9  require(msg.value > 0, "send ETH to deposit
10 rewards");
11 rewardsForA[_beneficiary] += msg.value;
12 }
13
14 function withdrawReward() public {
15 uint amountToWithdraw=rewardsForA[msg.sender];
16 require(amountToWithdraw > 0);
17 rewardsForA[msg.sender] = 0;
18 (bool success,)=msg.sender.call{value:
19 amountToWithdraw}("");
20 require(success);
21         }
22 }
23
```

(a) Safe Reentrancy_safe contract



(b) Augmented CFG with the infeasible reentrant path

Fig. 8. The safe Reentrancy_safe contract (a) and its augmented CFG (b). The reset `rewardsForA[ms]=0` precedes the external call, so the transfer guard fails (`rewardsForA[a]` $\mapsto [0,0]$); its true branch and the augmented (fallback) edge are infeasible ($\bot$), so no reentrant path exists.

Example 5.5. Consider the contract and its augmented CFG in Figure 9, which performs arithmetic operations on a state variable `count` of type `uint256`, using user inputs. To detect arithmetic anomalies, we apply fixpoint-based abstract interpretation using the interval domain. The initial abstract state $\overline{\mathcal{CS}}_{\text{init}}$ assigns the full `uint256` range $[0,\ 2^{256}-1]$ to `count` and all inputs. The fixpoint procedure (Algorithm 3) propagates interval constraints through the augmented CFG, computing updated intervals $[l_x, h_x]$ for each assignment of `count`. For addition, multiplication, and subtraction, the derived intervals are $[l_1 + l_2,\ h_1 + h_2]$, $[\min(\mathcal{P}),\ \max(\mathcal{P})]$ with $\mathcal{P} = \{l_1 \times l_3,\ l_1 \times h_3,\ h_1 \times l_3,\ h_1 \times h_3\}$, and $[l_1 - h_4,\ h_1 - l_4]$, respectively. Each result is checked against the `uint256` bounds $[0,\ 2^{256}-1]$. Violations occur when $h_x > 2^{256}-1$ (overflow) or $l_x < 0$ (underflow). In this example, the fixpoint semantics yield $h_1 + h_2 > 2^{256}-1$, $\max(\mathcal{P}) > 2^{256}-1$, and $l_1 - h_4 < 0$ for the respective paths, confirming integer overflow and underflow vulnerabilities.

5.3 Timestamp Dependency Vulnerability via Abstract Semantic Interpretation

Timestamp dependency vulnerabilities occur when contracts use mutable blockchain properties like *block.timestamp*, *block.number*, or *now*, to control execution or compute values. Although these parameters are bounded,

Just Accepted

Algorithm 3 Semantic Detection of Integer Overflow/Underflow via Interval Domain

Input: Augmented CFG $\mathcal{G} = (N, E)$, abstract domain $\mathcal{D} = \textsc{Interval}$
Output: Vulnerability report $\mathcal{V}$
$\overline{\mathcal{CS}} \leftarrow \textsc{FixpointCollectingSemantics}(\mathcal{G}, \mathcal{D})$ // Invoke Algorithm 1
Initialize $\mathcal{V} \leftarrow \emptyset$
for all assignment-node $n_i \in N$ **do**
  for all variable $v \in \text{def}(n_i)$ **do**
    $[l_x, h_x] \leftarrow \overline{Cx_i}(v)$
    $[l_{\min}, h_{\max}] \leftarrow \textsc{Range}(v)$
    if $h_x > h_{\max}$ **then**
      $\mathcal{V} \leftarrow \mathcal{V} \cup \{\text{Overflow } (n_i, v)\}$
    else if $l_x < l_{\min}$ **then**
      $\mathcal{V} \leftarrow \mathcal{V} \cup \{\text{Underflow } (n_i, v)\}$
    end if
  end for
end for
return $\mathcal{V}$

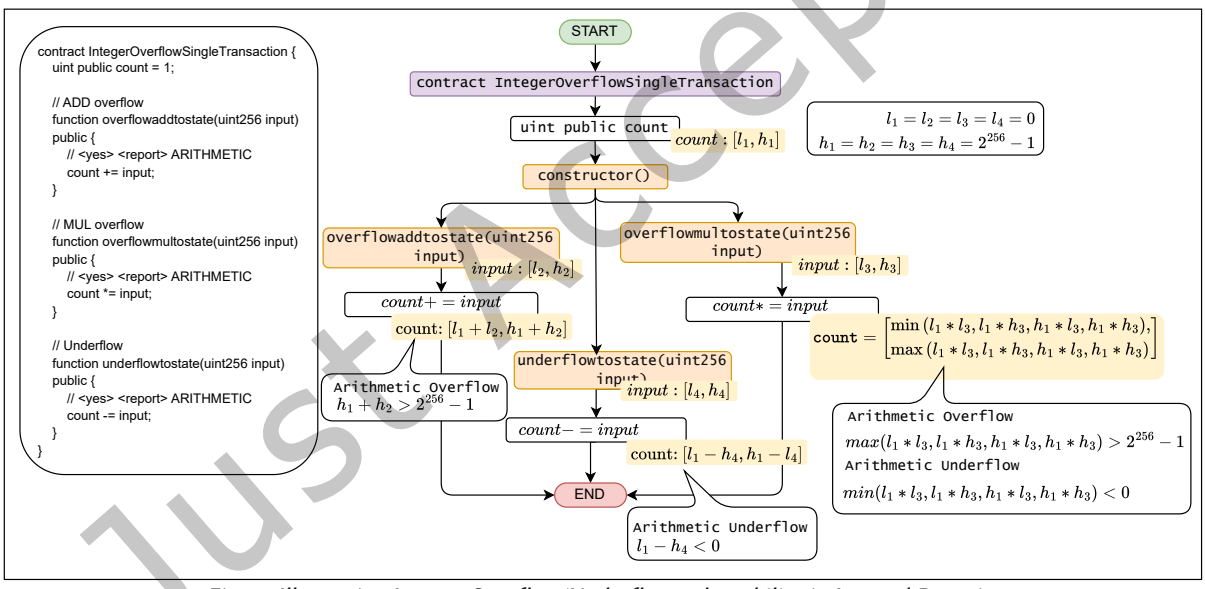


Fig. 9. Illustrating Integer Overflow/Underflow vulnerability in Interval Domain

Just Accepted

miners can manipulate them slightly to influence outcomes such as rewards or fund transfers. Detecting such vulnerabilities requires tracing timestamp propagation through data and control dependencies. Computation of such dependency via syntactic presence of the variables may often produce false positives. Semantics-based refinement of these dependencies, which considers values instead of variables, yields precision improvement and reduces the number of false dependencies. Our approach combines flow and context-sensitive syntax-based dependency analysis with semantic refinement using abstract interpretation.

5.3.1 Syntactic Dependency Computation. The computation of data and control dependencies in a program is initially derived from the syntactic structure of statements. We define below three different kinds of syntactic dependencies.

Definition 5.6 (Intra Data Dependency). Given an assignment statement id := exp at program point l, the intra-statement data dependency is defined as $\mathsf{id}' \rightarrow \mathsf{id}$, where id' are the variables present in exp. This indicates that the value assigned to the variable id directly depends on the value of the used variable id' within the same statement.

Definition 5.7 (Inter Data Dependencies). Given an augmented CFG, a node n_2 at program point l_2 is called data dependent on another node n_1 at program point l_1 denoted as $n_1 \xrightarrow{\mathsf{id}} n_2$, if there exists a variable id such that (1) id is defined by n_1, (2) id is used by n_2, and (3) there is a id definition free path from n_1 to n_2.

Definition 5.8 (Control Dependencies). Given an augmented CFG, node n_2 post-dominates node n_1 if $n_1 \neq n_2$ and every path from n_1 to *stop* contains n_2. Node n_2 is control dependent on node n_1 denoted by $n_1 \xrightarrow{\text{control}} n_2$, if there is a non-trivial path p from n_1 to n_2 such that n_2 post-dominates all nodes $n_3 \notin \{n_1, n_2\}$ on p and n_2 does not post-dominate n_1.

Example 5.9. Consider the code snippet in Figure 10. At program point 3, there exists an intra-statement data dependency w → v since w is used in the expression `v = 8 + ((4 * w) % 2)`. The node at point 5 is inter-statement data dependent on point 3, denoted $3 \xrightarrow{\mathsf{v}} 5$, as v is defined at 3 and used at 5 without redefinition along the path. Additionally, nodes 5 and 6 are control dependent on the condition at point 4, represented as $4 \xrightarrow{\text{control}} \{5, 6\}$, and the critical section at point 8 is control dependent on the guard at point 7, expressed as $7 \xrightarrow{\text{control}} 8$.

```
1 uint w = block.timestamp;
2 uint x = 0;
3 uint v = 8 + ((4 * w) % 2);
4 while (v >= 0){
5         x = v % 2;
6         v - = 2; }
7 if (x > 0){
8         // Critical Section
9 }
```

Fig. 10. Syntactic vs. semantic dependency.

To formalize dependency analysis, enabling its semantics based refinement over numerical abstract domain, we define a labeled transaction function $\overline{\mathcal{S}}$: stmt × Ψ where $\Psi \triangleq \mathit{Label} \times \mathit{Dep}$, where *Label* is a set of program points and *Dep* is the set of dependencies. The auxiliary functions F_{use} and F_{def} return the sets of used and defined variables, respectively. We use currLabel(stmt) and nextLabel(stmt) which return the label of stmt and the label of the statement following stmt respectively.

Assignment Statement. $\overline{\mathcal{S}}[\![^{\ell}\mathsf{id} := \mathsf{exp}]\!] = \{(\ell, \psi) \rightarrow (\ell', \psi_0) \mid \psi \in \Psi\}$ where $\ell' \in \mathsf{nextLabel}(^{\ell}\mathsf{id} := \mathsf{exp})$, $\psi_0 = \bigwedge\{\mathsf{id}_i \rightarrow \mathsf{id} \mid \mathsf{id}_i \in F_{\text{use}}(\mathsf{exp}) \wedge \mathsf{id}_i \neq \mathit{id}\} \bigwedge \{\mathsf{id}_j \rightarrow \mathsf{id}_k \mid \mathsf{id}_j \rightarrow \mathsf{id}_r, \mathsf{id}_r \rightarrow \mathsf{id}_k \in \psi\} \bigwedge (\psi \setminus \bigwedge\{\mathsf{id}_l \rightarrow \mathsf{id} \mid \mathsf{id}_l \notin F_{\text{use}}(\mathsf{exp})\})$.

Conditional.

$$\overline{\mathcal{S}}[\![\mathit{if}\,^{\ell}\mathsf{bexp}\ \mathit{then}\ \mathsf{stmt}_1\ \mathit{else}\ \mathsf{stmt}_2]\!] = \{(\ell, \psi) \rightarrow (\mathsf{currLabel}(\mathsf{stmt}_1), \psi)\} \cup \{(\ell, \psi) \rightarrow (\mathsf{currLabel}(\mathsf{stmt}_2), \psi)\} \cup \overline{\mathcal{S}}[\![\mathsf{stmt}_1]\!] \cup \overline{\mathcal{S}}[\![\mathsf{stmt}_2]\!] \cup \{(\ell, \psi) \rightarrow (\ell', \psi_1)\} \cup \{(\ell, \psi) \rightarrow (\ell', \psi_2)\}$$

where $\ell' \in \mathsf{nextLabel}(\mathit{if}\,^{\ell}\mathsf{bexp}\ \mathit{then}\ \mathsf{stmt}_1\ \mathit{else}\ \mathsf{stmt}_2\ \mathit{endif})$, and for $k \in \{1, 2\}$, we define ψ_k as: $\psi_k = \psi \wedge \bigwedge\{\mathsf{id}_i \rightarrow \mathsf{id}_j \mid \mathsf{id}_i \in F_{\text{use}}(\mathsf{bexp}) \wedge \mathsf{id}_j \in F_{\text{def}}(\mathsf{stmt}_k) \wedge \mathsf{id}_i \neq \mathsf{id}_j\}$

Iteration.

$$\overline{\mathcal{S}}[\![\mathit{while}\,^{\ell}\mathsf{bexp}\ \mathit{do}\ \mathsf{stmt}\ \mathit{done}]\!] = \{(\ell, \psi) \rightarrow (\mathsf{currLabel}(\mathsf{stmt}), \psi)\} \cup \overline{\mathcal{S}}[\![\mathsf{stmt}]\!] \cup \{(\ell, \psi) \rightarrow (\ell', \psi_1)\}$$

Just Accepted

where $\ell' \in \text{nextLabel}(\textit{while}\ ^{\ell}\text{bexp}\ \textit{do}\ \text{stmt}\ \textit{done})$, and $\psi_1 = \psi \wedge \bigwedge\{\text{id}_i \rightarrow \text{id}_j \mid \text{id}_i \in F_{\text{use}}(\text{bexp}) \wedge \text{id}_j \in F_{\text{def}}(\text{stmt}) \wedge \text{id}_i \neq \text{id}_j\}$

Constructor.

$$\overline{\mathcal{S}}[\![^{\ell}\text{constructor(par)}\{\text{stmt}\}]\!] = \{(\ell, \psi) \rightarrow (\text{currLabel(stmt)}, \psi)\} \cup \overline{\mathcal{S}}[\![\text{stmt}]\!]$$

Function.

$$\overline{\mathcal{S}}[\![^{\ell}\textit{function}\ \text{id}(\text{par}_{\text{in}})\ (\textit{returns}(\text{par}_{\text{out}}))\{\text{stmt}\}]\!] = \{(\ell, \psi) \rightarrow (\text{currLabel(stmt)}, \psi)\} \cup \overline{\mathcal{S}}[\![\text{stmt}]\!]$$

Smart Contract.

$$\begin{aligned}\overline{\mathcal{S}}[\![^{\ell}\textit{contract}\ \text{id}\{\text{sdecl constr? func}^*\}]\!] = \{((\ell', \psi) \rightarrow (\ell'', \psi')) \in \overline{\mathcal{S}}[\![\text{constr?}]\!]\} \\ \cup \{((\ell'', \psi') \rightarrow (\ell', \psi'')) \in \overline{\mathcal{S}}[\![\text{func}^*]\!]\}\end{aligned}$$

5.3.2 Semantics-based Dependency Refinement. We now employ semantics refinement at two different levels: intra statement (expression level) and inter statement (statement level), inspired by [21, 22, 31]. Intra-statement refinement allows to identify false intra data dependencies, whereas inter-statement refinement allows to identify false inter data dependency.

Definition 5.10 (Expression level Semantic Dependencies). Given an expression exp, let $\text{id} \in \text{Var(exp)}$. The expression exp is called semantically dependent on id, denoted by $\text{id} \rightsquigarrow \text{exp}$, iff $\exists\, \sigma_1, \sigma_2 \in \Sigma.\ \forall\, \text{id}' \neq \text{id},\ \sigma_1(\text{id}') = \sigma_2(\text{id}') \wedge \mathcal{S}[\![\text{exp}]\!](\sigma_1) \neq \mathcal{S}[\![\text{exp}]\!](\sigma_2)$. This definition enables us to formally identify the subset of variables, denoted by Vars(exp), that are essential for determining the outcome of evaluating expression exp under a given program state.

Definition 5.11 (Statement level Semantic Relevancy). Given a smart contract sc and a concrete property ω on states, the statement stmt at program point ℓ in sc is semantically relevant *w.r.t.* ω if: $\exists \sigma \in \Sigma_\ell : \mathcal{S}[\![\text{stmt}]\!](\sigma) = \sigma' \wedge \omega(\sigma) \neq \omega(\sigma')$, where Σ_ℓ are the set of states that can possibly reach the program point ℓ. In other words, the statement stmt at program point ℓ is semantically irrelevant *w.r.t.* a concrete property ω if the execution of stmt over any state $\sigma \in \Sigma_\ell$ yields to a state that is equivalent to σ *w.r.t.* ω. Whenever ω distributes over program variables, we define it as follows:

Definition 5.12 (Concrete Semantic Relevancy for a Set of Variables). Given a smart contract sc and a concrete property ω that distributes over program variables, the statement *stmt* at program point ℓ in sc is semantically relevant *w.r.t.* ω for a subset of variables V if : $\exists \sigma = \langle \sigma(v_1), \sigma(v_2), \ldots, \sigma(v_k)\rangle \in \Sigma_\ell$ *and* $\exists v_i \in V$such that $\omega(\pi_i(\mathcal{S}[\![stmt]\!]\sigma)) \neq \omega(\pi_i(\sigma))$, where $\pi_i(\langle \sigma(v_1), \sigma(v_2), \ldots, \sigma(v_k)\rangle) = \sigma(v_i)$.

We now lift these notions of semantic refinement from concrete to abstract domains representing the properties of interest.

Definition 5.13 (Expression Level Abstract Semantic Dependencies). Given an expression exp and an abstract domain $\mathcal{D}$, let $\text{id} \in \text{Var(exp)}$. The expression exp is called semantically dependent on id *w.r.t.* $\mathcal{D}$, denoted by $\text{id} \rightsquigarrow_{\mathcal{D}} \text{exp}$, iff : $\exists\, \overline{\sigma_1}, \overline{\sigma_2} \in \overline{\Sigma} :$ $\text{ATOM-STATE}(\overline{\sigma_1}) \wedge \text{ATOM-STATE}(\overline{\sigma_2}) \wedge \forall\, \text{id}' \neq \text{id}.\ \overline{\sigma_1}(\text{id}') = \overline{\sigma_2}(\text{id}') \wedge \neg\, \text{ATOM-VAL}\big(\overline{\mathcal{S}}[\![\text{exp}]\!](\overline{\sigma_1} \cup \overline{\sigma_2})\big)$, where ATOM-STATE map each variable to atomic value of the abstract domain and ATOM-VAL denotes the atomic value of the abstract domain. The non-atomicity requirement on the evaluations of exp implies that all concrete evaluations (which yield singletons) over $\sigma_1 \cup \sigma_2$ are not abstracted to the same abstract value. That is, $\overline{\sigma_1}$ and $\overline{\sigma_2}$ may result in different values for exp.

Definition 5.14 (Statement Level Abstract Semantic Relevancy). Given a smart contract sc and a statement $\text{stmt} \in \text{sc}$ at program point ℓ, let $\overline{Cx_\ell}$ be the collecting semantics at ℓ. The statement stmt is called semantically

Just Accepted

relevant *w.r.t.* abstract property $\mathcal{D}$ if: $\exists\, \text{stmt}_\ell \in \text{pred}(\text{stmt})\ ,\ \exists\, \overline{\sigma} \in \overline{Cx_\ell}\ :\ \overline{\mathscr{S}}[\![\text{stmt}]\!]\,\overline{\sigma} \neq \overline{\sigma}$. In other words, the statement stmt at program point ℓ is semantically irrelevant *w.r.t.* an abstract property $\mathcal{D}$ if no changes take place in the abstract states $\overline{\sigma} \in \overline{Cx_\ell}$ for all $\text{stmt}_\ell \in \text{pred}(\text{stmt})$, when stmt is executed over $\overline{\sigma}$.

Definition 5.15 (Abstract Semantic Relevancy for a Set of Variables). Given a smart contract sc and a statement *stmt* $\in$ sc at program point ℓ, let $\overline{Cx_\ell}$ be the collecting semantics at ℓ. The statement *stmt* is called semantically relevant *w.r.t.* abstract property $\mathcal{D}$ for a subset of variables V if $\exists\, \overline{\sigma} = \langle \mathcal{D}(v_1), \mathcal{D}(v_2), \ldots, \mathcal{D}(v_k)\rangle \in \overline{Cx_\ell},\ \exists\, v_i \in V$ such that $\pi_i\left(\overline{\mathscr{S}}[\![\text{stmt}]\!]\overline{\sigma}\right) \neq \pi_i(\overline{\sigma})$.

Example 5.16. Consider the code snippet shown in Figure 10, where v is computed as $v = 8 + ((4 * w)\%2)$, with $w = block.timestamp$. Since $(4 \times w)$ is divisible by 2 for all values of w, the expression $((4 \times w) \bmod 2)$ always evaluates to 0. Consequently, v always evaluates to 8, regardless of the value of w. This demonstrates the motivation behind semantic independence. According to our abstract analysis (Definition 5.13), v is expression-level semantically independent of w, even though the syntactic dependency $w \rightarrow v$ remains present.

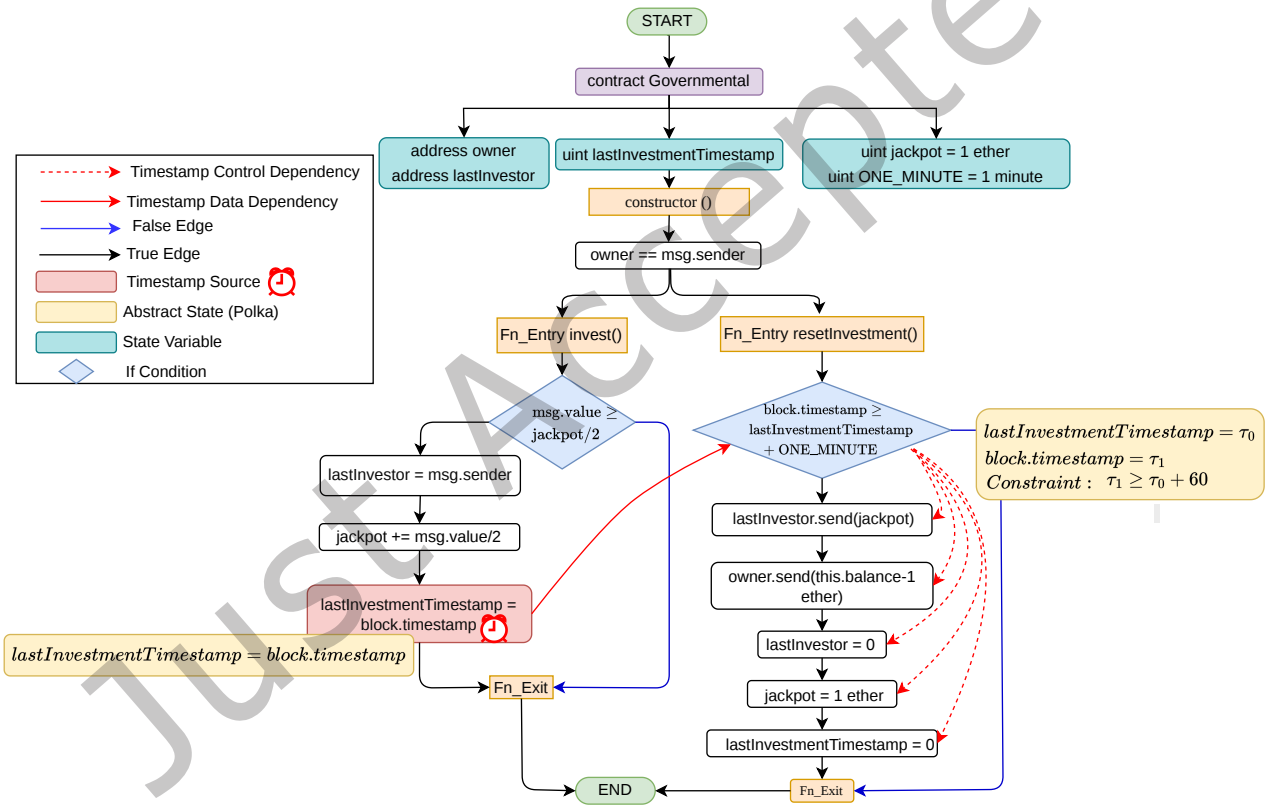


Fig. 11. Illustrating timestamp dependency vulnerability.

Example 5.17. Recall the code snippet shown in Figure 2b. The contract features a one-minute delayed payout to prior investors. The corresponding augmented control flow graph (CFG), annotated with timestamp data and control dependencies and abstract interpretation results, is shown in Figure 11. The `invest()` function assigns the current block timestamp to the state variable `lastInvestmentTimestamp`. The `resetInvestment()`

Just Accepted

function later checks whether at least one minute has elapsed (`block.timestamp < lastInvestmentTimestamp + ONE_MINUTE`) before permitting an Ether transfer. First, our analysis reveals a syntactic data dependency from `block.timestamp` to `lastInvestmentTimestamp`, and a syntactic control dependency from the `resetInvestment()` guard to the payout, marking both branches as timestamp-influenced. Annotated edges in the augmented CFG represents these dependencies. After that, we compute the contract's abstract semantics in the polyhedra domain, which confirms the semantic feasibility of the execution path and captures the constraint $\tau_1 \geq \tau_0 + 60$ (where τ_0 and τ_1 are the timestamps for investment and reset, respectively). Given the slight timestamp adjustments allowed for Ethereum miners, this path is practically feasible, making the contract logic susceptible to adversarial transaction ordering.

5.4 Transaction Ordering Dependency via Abstract Semantic Interpretation

Transaction Ordering Dependency (TOD), also known as front-running, allows malicious actors (the contract owner or miners) to manipulate the transaction order within the same block to gain unfair advantages. Our framework involves a two-phase approach to detect TOD by combining static dataflow analysis followed by semantic refinement via abstract interpretation. Using this approach, we not only capture the inter-procedural data flows but can also handle subtle intra-procedural dependencies that existing tools often miss.

Algorithm 4 Transaction Ordering Dependency (TOD) Detection

1: **Input:** Augmented CFG $\mathcal{G} = (N, E)$, abstract domain $\mathcal{D}$
2: **Output:** Set $\mathcal{V}_{\text{TOD}}$ of confirmed TOD vulnerabilities
3: $\overline{\mathcal{CS}} \leftarrow$ FixpointCollectingSemantics($\mathcal{G}, \mathcal{D}$) // Invoke Algorithm 1
4: **for all** sensitive state variables v **do**
5: Let $F_{\text{def}}(v)$ be functions that define v
6: Let $F_{\text{use}}(v)$ be functions that use v
7: **for all** $(f_d^v, f_u^v) \in F_{\text{def}}(v) \times F_{\text{use}}(v)$ **do**
8: **if** checkRelevancy($f_{\text{def}}^v, v, \overline{\mathcal{CS}}$) and checkDependency($f_{\text{use}}^v, v, \overline{\mathcal{CS}}$) and checkConditionDependency($f_{\text{def}}^v, f_{\text{use}}^v, v, \overline{\mathcal{CS}}$) **then**
9: $\mathcal{V}_{\text{TOD}} \leftarrow \mathcal{V}_{\text{TOD}} \cup \{(v, f_{\text{def}}^v, f_{\text{use}}^v)\}$
10: **end if**
11: **end for**
12: **end for**
13: **return** $\mathcal{V}_{\text{TOD}}$

Algorithm 4 outlines the detection process of TOD Vulnerability. In this algorithm, first compute the abstract collecting semantics $\overline{\mathcal{CS}}$ of the contract using the FixpointCollectingSemantics (Step 3) of Algorithm 1. This semantic computation allows for accurate reasoning over value flows and control conditions among interleaved transactions. The algorithm finds the set of functions $F_{\text{def}}(v)$ that define each sensitive state variable v (Step 5), and the set of functions $F_{\text{use}}(v)$ that use v (Step 6). These functions may belong to the same procedure (intra-procedural case) or different procedures (inter-procedural case). The algorithm then iterates over all function pairs $(f_{\text{def}}^v, f_{\text{use}}^v)$ such that $f_{\text{def}}^v \in F_{\text{def}}(v)$ and $f_{\text{use}}^v \in F_{\text{use}}(v)$ (Step 7), and applies a conjunction of semantically guided checks to assess whether the pair may yield a feasible TOD vulnerability. The predicate checkRelevancy validates whether the assignment to v in f_{def}^v influences any downstream abstract states in $\overline{\mathcal{CS}}$, effectively filtering out definitions that are semantically irrelevant, i.e., definitions such that $s_{\text{def}}^v \in f_{\text{def}}^v$ does not affect the program's observable semantics. The predicate checkDependency verifies whether the use of v in f_{use}^v has a semantically meaningful

Just Accepted

impact on control flow or security-critical behavior. In particular, it eliminates those use sites $s^{v}_{\text{use}} \in f^{v}_{\text{use}}$ where the outcome is semantically independent of v, as determined from the abstract semantics. These two checks ensure that both the definition and the use of v contribute to observable behavioral differences under transaction reordering. This means that reordering the transactions can change the outcome only if the write semantically changes v (otherwise, v remains the same in either order) and the read semantically depends on v (otherwise, the result is the same for every value of v). After this, checkConditionDependency checks that the execution of f^{v}_{use} depends on the value of v that is defined in f^{v}_{def}. This reliance is important for finding any semantic divergence when transactions are reordered in a hostile way, which is a key feature of TOD vulnerabilities. If all three predicates checkRelevancy, checkDependency, and checkConditionDependency are satisfied (Step 8), the variable-function triple $(v, f^{v}_{\text{def}}, f^{v}_{\text{use}})$ is classified as vulnerable.

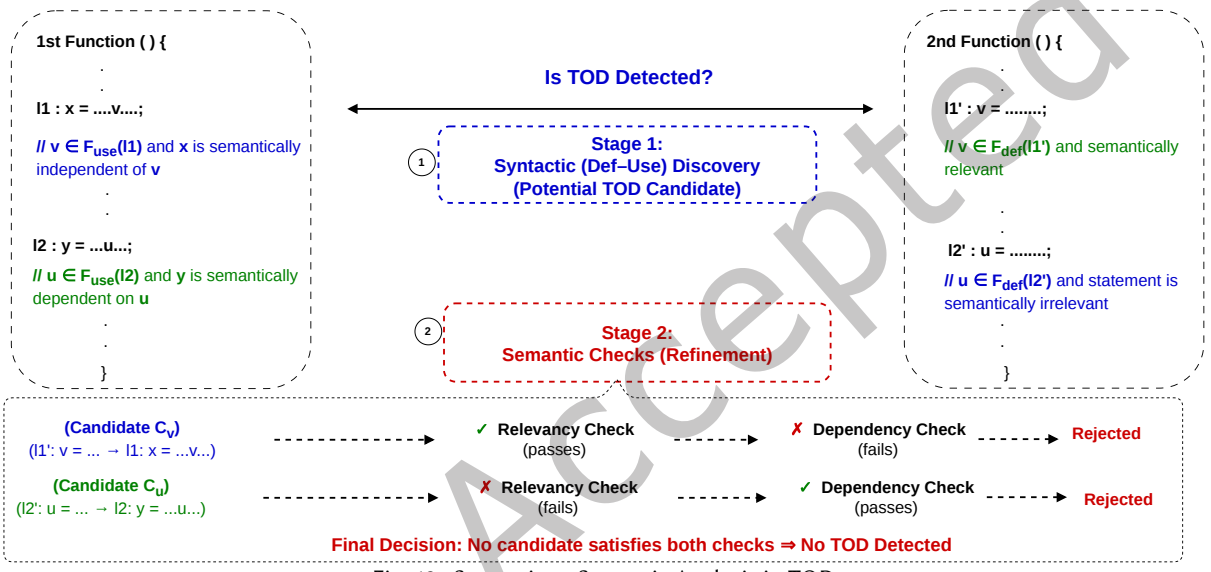


Fig. 12. Syntactic vs Semantic Analysis in TOD

The schematic diagram in Figure 12 contrasts a purely syntactic TOD check with our semantic analysis. The first function reads the state variables v and u defined by the second function, and uses them to compute x and y. This syntactic check determines def-use links and reports a possible TOD. However, the semantic analysis tests each pair against the abstract collecting semantics (Algorithm 1) and observes the following. For v, the definition is relevant, but x is semantically independent of v (Definition 5.13), so checkDependency fails. For u, the definition of y semantically depends on u, but the statement defining u is semantically irrelevant (Definition 5.14), so checkRelevancy fails. Each candidate pair is thus eliminated through semantic check, and hence no TOD is reported.

Example 5.18. Recall the TOD vulnerability in the `EGame` contract shown earlier in Figure 2d. Figure 13 illustrate how our framework detects the vulnerability through the two-phase analysis. The analysis finds the sensitive state variable `winner`, in the first phase. This variable is defined in the `play()` function and used in the `getReward()` function. The corresponding function pair (`play`, `getReward`) is selected based on the presence of a def-use chain over `winner`. The dashed arrow shows the critical interprocedural flows involving this variable. The predicate checkRelevancy in the second phase of the analysis confirms that the assignment to `winner` in `play()` is semantically relevant, as it influences the observable transfer behavior in `getReward()`. The predicate

Just Accepted

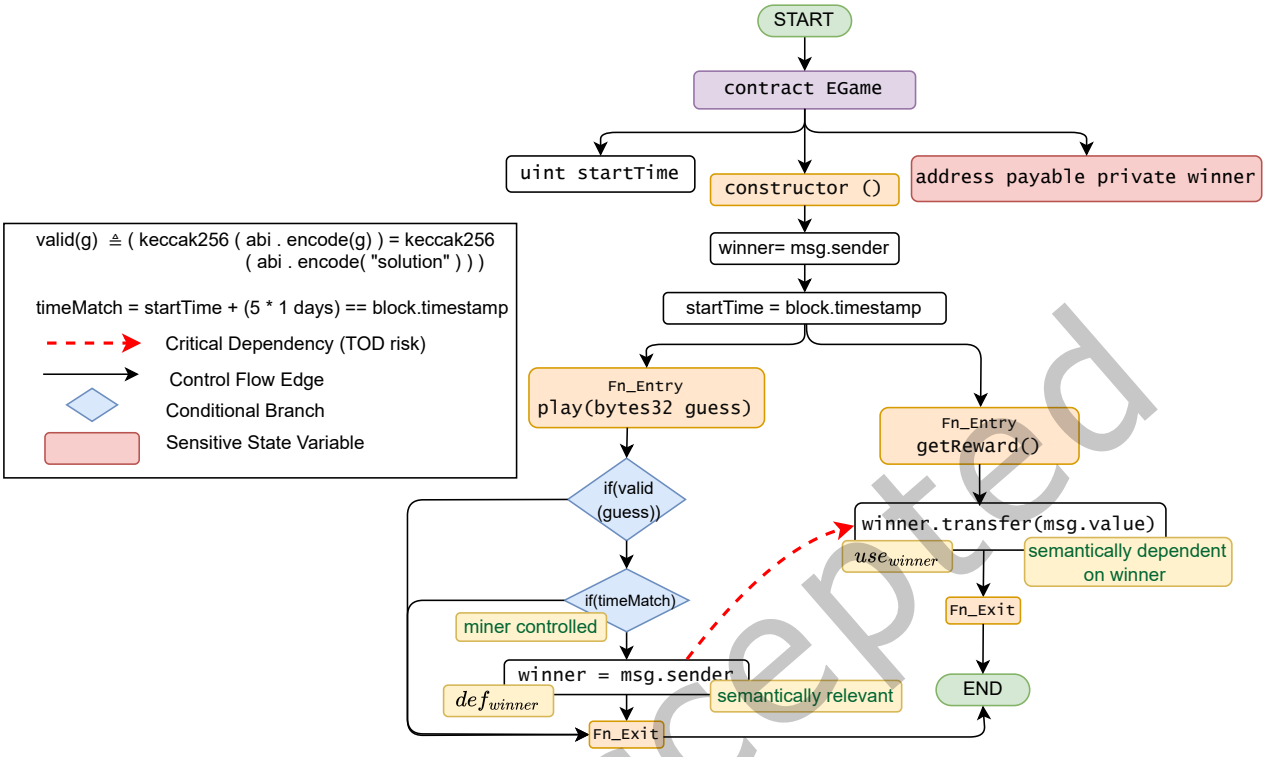


Fig. 13. Illustrating TOD vulnerability

checkDependency determines that the transfer operation in `getReward()` semantically depends on the value of `winner`, thereby establishing a meaningful use. Finally, checkConditionDependency confirms that the update to `winner` in `play()` is gated by a timestamp condition that is affected by miners, which makes it possible for transactions to be reordered in a bad way. Since all three predicates evaluate to true, the variable-function triple (`winner`, `play`, `getReward`) is reported as a confirmed TOD vulnerability.

6 Prototype Implementation

Our analysis tool [1] follows a modular, layered architecture that separates Solidity parsing, control-flow construction, dependency analysis, and abstract semantics evaluation, all implemented in Python, as illustrated in Figure 14. The tool takes input from a Solidity smart contract (.sol file). It compiles it using the `solcx` interface, invoking the appropriate version of the `solc` compiler as selected via a dynamic version manager (`SolcSelector`). We normalize the resulting AST into an intermediate JSON representation, where preprocessing modules convert Solidity-specific constructs, such as `mapping`, `enum`, and `struct`, into semantically equivalent forms that the Apron abstract domains can process. In this prototype, we cover the language constructs that our analyses examine in the benchmark contracts used in our evaluation. Flat mappings of the form `mapping(address => uint)` are projected to scalar variables, `require` statements are encoded as conditionals, and inline-assembly blocks are skipped with a diagnostic rather than interpreted. This does not limit the applicability of the tool to

[1] The complete source code is available at https://github.com/maitrikushwaha/solidity-analysis-framework-tool

Just Accepted

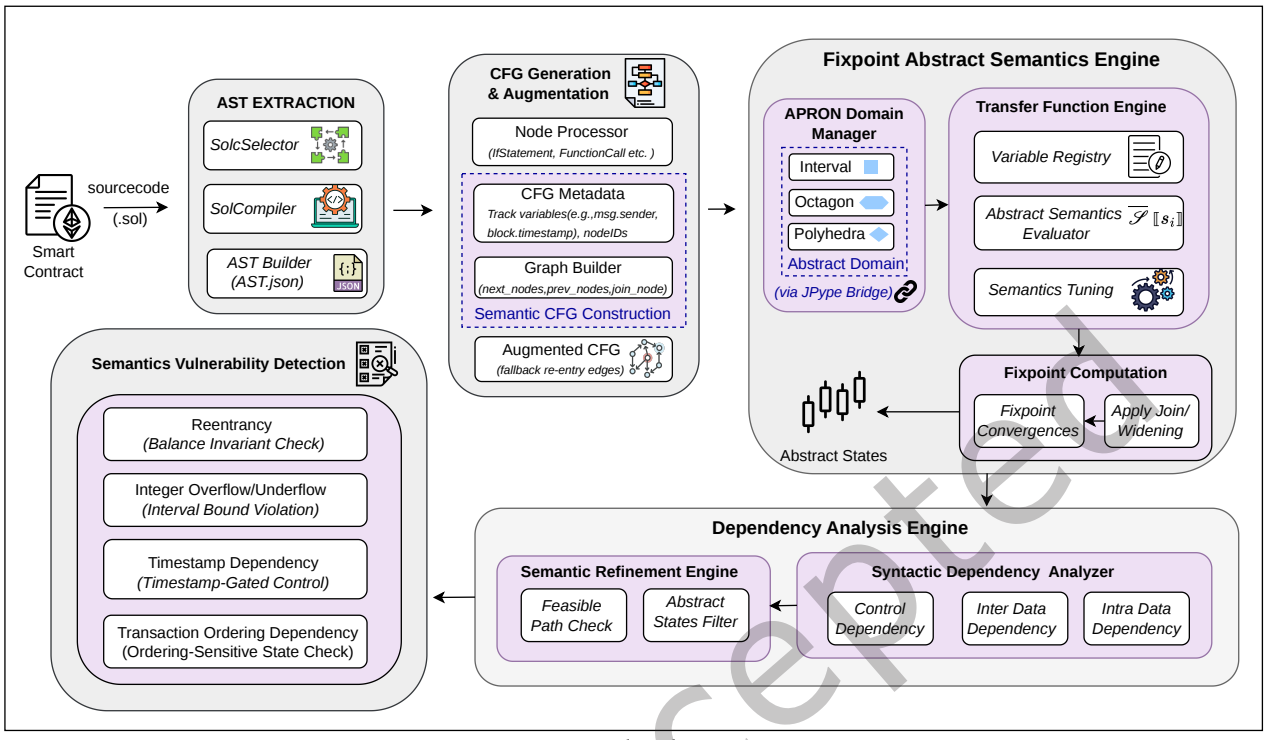


Fig. 14. Tool Architecture

real-world smart contract vulnerability detection within the scope of this paper. During control-flow construction, the NodeProcessor recursively dispatches AST nodes to statement-specific handlers (e.g., `IfStatement`, `FunctionCall`). The `CFGBuilder` creates typed nodes with interprocedural entry/exit points, synthetic join nodes for conditionals and loops, and symbolic start/end anchors. All node metadata, including IDs, variable usage, and blockchain-specific identifiers like `msg.sender` and `block.timestamp`, is tracked via `CFGMetadata`. After constructing the Semantics CFG, we augment the graph by inserting fallback edges from external call sites (`call`, `delegatecall`, `transfer`) to function entry points that a reentrant caller can invoke during execution.

Before the core analysis begins, the `VariableRegistry` maps each program variable (particularly state variables) to the numerical dimensions required by the `APRON` library. The Abstract semantics evaluator translates Solidity expressions into APRON's symbolic tree format using `apron.Texpr0Intern` class. We then run the `Fixpoint Abstract Semantics Engine` on this augmented CFG to compute collecting semantics by iteratively propagating abstract states across nodes using statement-specific transfer functions. The transfer functions work on numerical domains (like intervals, octagons, and polyhedra) handled by the `APRON Domain Manager`. A `JPype` bridge connects our Python prototype to these native libraries, which lets us efficiently handle symbolic state updates and domain operations. The engine is tuned semantically by choosing a suitable APRON numerical domain to balance precision and efficiency. The Dependency Analysis Engine is responsible to detect semantically feasible paths along the augmented CFG of the contract. In particular, it runs the following two phases: first, the Syntactic Dependency Analyzer computes the intra-statement, inter-statement, and control dependencies over the augmented CFG, as defined in Section 5.3.1. The Semantic Refinement Engine then uses the fixpoint abstract states from Algorithm 1 to discard the false dependencies, following the refinement criteria of Section 5.3.2. This

Just Accepted

Table 2. Overview of the four benchmark datasets. **Solidity Version Range** gives the range of declared pragma versions in each contract set. **Vulnerability Present** and **Vulnerability Absent** report the numbers of contracts with and without the target vulnerability, respectively. **Total** gives the number of evaluated contracts, and **LoC** gives their total source lines of code.

Dataset	Vulnerability	Solidity Version Range	Vulnerability Present	Vulnerability Absent	Total	LoC
SBC	Reentrancy	0.4.0–0.5.0	31	111	142	14,416
	Integer Overflow/Underflow		15	127		
	Timestamp Dependency		14	128		
	Transaction-Ordering Dependency		4	138		
Qian	Reentrancy	0.4.25	69	153	222	3,346
	Integer Overflow/Underflow	0.4.25	86	189	275	4,287
	Timestamp Dependency	0.4.25	173	176	349	5,079
RSD	Reentrancy	0.8.20	66	72	138	5,068
SolidiFI	Transaction-Ordering Dependency	0.4.21–0.5.11	50	50	100	45,166

way the engine discards paths that are semantically unreachable in the chosen domain. This enables precise detection of both timestamp dependency and TOD vulnerabilities in the contract, as detailed in Sections 5.3 and 5.4.

7 Experimental Analysis

Datasets. We evaluate our proposal on four publicly available datasets: the SmartBugs Curated dataset (SBC) [17], the dataset of Qian et al. (Qian) [39], the Reentrancy Scenarios Dataset (RSD) [40], and the SolidiFI bug-injection benchmark (SolidiFI) [20]. We consider four vulnerability classes: reentrancy, integer overflow and underflow, timestamp dependency, and transaction-ordering dependency (TOD). Table 2 reports the Solidity version range, the numbers of contracts in which the vulnerability is present or absent, the total number of evaluated contracts, and the total source lines of code (LoC). As observed, the RSD dataset covers modern Solidity features and diverse reentrancy patterns, while the SolidiFI dataset is used for the TOD evaluation. During preprocessing, we manually inspect the SBC dataset and identify nine additional contracts exhibiting timestamp dependency that are located outside the corresponding subfolder. We also find one duplicate entry across the reentrancy and unchecked low-level call categories. After preprocessing, the final dataset for timestamp category contains 14 distinct contracts. We also exclude contracts whose vulnerability patterns fall outside the scope of our numerical abstract domains. In particular, reentrancy patterns that cannot be expressed as numerical Ether-balance properties, such as delegated reentrancy, non-Ether callback reentrancy, and non-address-keyed accounting state, are outside the scope of the current analysis. These exclusions are applied before running the tools, and the remaining contract sets are used for all tools. All results are reported at the contract level. For SBC, we use a one-vs-rest strategy in which contracts containing the target vulnerability form the positive class, while all remaining contracts form the negative class. For Qian, RSD, and SolidiFI, we use the contract sets provided for each evaluated vulnerability class, separating contracts in which the vulnerability is present from those in which it is absent.

Just Accepted

Table 3. Vulnerability classes supported by the evaluated tools (✓: supported; —: not supported). Tool versions are shown in parentheses where available.

Tool	Reentrancy	Integer Overflow/Underflow	Timestamp Dependency	TOD
Mythril (v0.24.8)	✓	✓	✓	✓
Oyente+	✓	✓	✓	✓
Slither (v0.10.4)	✓	—	✓	—
Osiris	✓	✓	✓	✓
SmartCheck (v2.0.3)	✓	—	✓	—
Sailfish	✓	—	—	✓
Vandal	✓	—	—	—
EtherSolve (v1.1)	✓	—	—	—
Our Tool	✓	✓	✓	✓

Tool Selection. To evaluate our framework, we select eight publicly available state-of-the-art security analysis tools: Mythril, Oyente+, Slither, Osiris, SmartCheck, Sailfish, Vandal, and EtherSolve. The tools are selected according to the following criteria:

- **Public availability.** The source code or executable environment of each tool is publicly available, and the tool supports at least one of the vulnerability classes considered in this study.
- **Established use in prior research.** The selected tools have been widely used in smart-contract security studies and comparative evaluations [8, 39, 51].
- **Reproducible execution.** We use a fixed version and execution environment for each tool. All baselines except Mythril are run in pinned Docker environments, while Mythril is run from its official release. The exact image digests, source commits, and execution settings are reported in Section 7.3.
- **Coverage of analysis approaches.** The selected tools cover the commonly used approaches in smart-contract vulnerability detection, including symbolic execution, source-level data-flow analysis, pattern matching, taint analysis, dependency analysis, logic-based analysis, and bytecode control-flow analysis.

Table 3 summarizes the vulnerability classes supported by the selected tools.

Abstract-domain configuration. Our prototype uses the APRON library [24] with three alternative numerical abstract domains: Interval (`Box`), Octagon, and Polyhedra (`Polka`). Each analysis run uses one domain, selected automatically according to the vulnerability class. The default setup uses the Interval domain for integer overflow/underflow and reentrancy, as these analyses primarily rely on per-variable and balance range abstractions. Similarly, the Polyhedra domain is used for the semantic refinement of timestamp and transaction-ordering dependency candidates because it preserves relations among numerical program and blockchain variables more precisely. The Octagon domain provides an intermediate relational abstraction. For the domain comparison in RQ2, each benchmark contract is analyzed separately under all three domains. The Interval analysis applies the threshold-based widening operator in Definition 5.2 to ensure fixpoint termination. All domain and widening settings are fixed across the benchmarks, with no per-contract manual tuning.

Evaluation Metrics. We use standard evaluation metrics to compare the detection efficacy of the analyzed tools. Let TP, FP, FN, and TN denote the numbers of true positives, false positives, false negatives, and true negatives,

Just Accepted

respectively. The metrics Precision (P), Recall (R), F1 score ($F1$), False Discovery Rate (FDR), and False Negative Rate (FNR) are defined as follows: $\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP+FP}}$, $\mathrm{R} = \frac{\mathrm{TP}}{\mathrm{TP+FN}}$, $\mathrm{F1} = 2 \times \frac{\mathrm{P \times R}}{\mathrm{P+R}}$, $\mathrm{FDR} = \frac{\mathrm{FP}}{\mathrm{TP+FP}}$, $\mathrm{FNR} = \frac{\mathrm{FN}}{\mathrm{TP+FN}}$.

7.1 Performance Evaluation

7.1.1 Effectiveness of Detecting Vulnerability. We evaluate our framework against eight baseline tools on four vulnerability classes across the SBC, Qian, RSD, and SolidiFI benchmarks. Since not every baseline supports every vulnerability class, each comparison includes only the applicable tools. Tables 4–7 present the detailed results for the four vulnerability classes. In these tables, *Analyzed* denotes the number of contracts included in the TP, FP, TN, and FN counts, while *Total* denotes the benchmark size. The *Failures* column reports the number of contracts for which a tool does not complete the analysis because of unsupported constructs, compilation errors, or timeouts. These contracts are excluded from the TP, FP, TN, and FN counts. For our tool, these failures arise from unsupported constructs that prevent the preprocessing stage from constructing the augmented CFG required for the analysis. P, R, F1, FDR, and FNR are reported as percentages.

Reentrancy. As reported in Table 4, our framework obtains the highest F1 score on all three reentrancy benchmarks: 88.2% on SBC, 92.6% on Qian, and 85.2% on RSD. It detects every vulnerable contract within the analyzed sets, yielding 100% recall and no false negatives. On SBC, our framework analyzes 30 of the 31 reentrancy-positive contracts and detects all 30. The remaining positive contract, `spank_chain_payment.sol`, is reported under *Failures* because it involves multiple inheritance, which our tool is currently unable to process, as noted in Section 7.4. The remaining three failed cases belong to the reentrancy-negative class. Oyente+ provides the closest baseline result on SBC, with an F1 score of 84.1%, while Sailfish attains higher precision but misses eight vulnerable contracts. On Qian, Slither performs competitively with an F1 score of 91.3%, but our framework detects the remaining vulnerable contract while maintaining comparable precision. The improvement is more pronounced on RSD, where the best baseline F1 score is 74.7% for Slither. Osiris and Sailfish do not analyze the RSD contracts, while EtherSolve detects none of the vulnerable contracts and SmartCheck records only 7.6% recall. These results show that the balance preservation analysis maintains high recall across both legacy and modern reentrancy benchmarks.

Integer overflow and underflow. Table 5 summarizes the integer overflow and underflow detection. On SBC, our framework achieves the highest F1 score of 54.5%, only 1.2 percentage points above Mythril. This result is driven by complete recall: our framework detects all 15 vulnerable contracts, whereas Mythril, Oyente+, and Osiris miss three, two, and two contracts, respectively. However, the 25 false positives produced by our analysis limit its precision to 37.5%. This result reflects the strongly imbalanced one-vs-rest setting, in which only 15 of the 142 SBC contracts belong to the arithmetic-positive class. The difference is substantially larger on Qian. Our framework detects all 86 vulnerable contracts with only nine false positives, resulting in 90.5% precision and an F_1 score of 95.0%. The strongest baseline, Osiris, reaches an F1 score of 66.2%. Mythril obtains slightly higher precision at 92.0%, but detects only 23 of the 86 vulnerable contracts. These results show that fixpoint-based range analysis can retain high recall while accurately checking arithmetic values against their declared Solidity type bounds.

Timestamp dependency. Table 6 shows that the timestamp results differ substantially between SBC and Qian. On SBC, Mythril achieves the highest F1 score of 85.7%. Our framework and Slither both obtain 75.7%. Our framework detects all 14 vulnerable contracts but reports nine false positives, whereas Mythril trades two false negatives for a lower false-positive count. Thus, complete recall does not produce the highest F1 score on this small and imbalanced benchmark. On Qian, our framework achieves 100% recall, 85.6% precision, and an F1 score of 92.3%, compared with 55.0% for the next-best baseline, Slither. In particular, our framework reports no false negatives, while Mythril, Oyente+, Slither, Osiris, and SmartCheck miss 114, 172, 63, 164, and 171 vulnerable contracts, respectively. The result indicates that propagating timestamp-derived values through program state

Just Accepted

Table 4. Reentrancy detection results.

Tool	TP	FP	TN	FN	Analyzed	Total	Failures	P	R	F1	FDR	FNR
SBC												
Ours	30	8	100	0	138	142	4	79.0	100.0	**88.2**	21.1	0.0
Mythril	29	39	72	2	142	142	0	42.6	93.5	58.6	57.4	6.5
Oyente+	29	9	101	2	141	142	1	76.3	93.5	84.1	23.7	6.5
Slither	29	28	78	0	135	142	7	50.9	100.0	67.4	49.1	0.0
Osiris	26	23	68	0	117	142	25	53.1	100.0	69.3	46.9	0.0
SmartCheck	29	56	55	2	142	142	0	34.1	93.5	50.0	65.9	6.5
Sailfish	23	4	104	8	139	142	3	85.2	74.2	79.3	14.8	25.8
Vandal	20	48	63	11	142	142	0	29.4	64.5	40.4	70.6	35.5
EtherSolve	30	56	54	1	141	142	1	34.9	96.8	51.3	65.1	3.2
Qian												
Ours	69	11	142	0	222	222	0	86.2	100.0	**92.6**	13.8	0.0
Mythril	36	60	93	33	222	222	0	37.5	52.2	43.6	62.5	47.8
Oyente+	61	73	80	8	222	222	0	45.5	88.4	60.1	54.5	11.6
Slither	68	12	140	1	221	222	1	85.0	98.6	91.3	15.0	1.5
Osiris	64	139	14	5	222	222	0	31.5	92.8	47.1	68.5	7.2
SmartCheck	62	110	43	7	222	222	0	36.0	89.9	51.4	63.9	10.1
Sailfish	46	2	151	23	222	222	0	95.8	66.7	78.6	4.2	33.3
Vandal	62	137	16	7	222	222	0	31.2	89.9	46.3	68.8	10.1
EtherSolve	65	82	71	4	222	222	0	44.2	94.2	60.2	55.8	5.8
RSD												
Ours	66	23	49	0	138	138	0	74.2	100.0	**85.2**	25.8	0.0
Mythril	56	44	26	10	136	138	2	56.0	84.9	67.5	44.0	15.2
Oyente+	41	30	41	24	136	138	2	57.8	63.1	60.3	42.2	36.9
Slither	62	38	34	4	138	138	0	62.0	93.9	74.7	38.0	6.1
Osiris	NA	NA	NA	NA	NA	138	NA	NA	NA	NA	NA	NA
SmartCheck	5	12	60	61	138	138	0	29.4	7.6	12.0	70.6	92.4
Sailfish	NA	NA	NA	NA	NA	138	NA	NA	NA	NA	NA	NA
Vandal	61	51	19	4	135	138	3	54.5	93.8	68.9	45.5	6.2
EtherSolve	0	0	72	66	138	138	0	–	0.0	0.0	–	100.0

Just Accepted

Table 5. Integer overflow and underflow detection results.

Tool	TP	FP	TN	FN	Analyzed	Total	Failures	P	R	F1	FDR	FNR
SBC												
Ours	15	25	98	0	138	142	4	37.5	100.0	**54.5**	62.5	0.0
Mythril	12	18	109	3	142	142	0	40.0	80.0	53.3	60.0	20.0
Oyente+	13	101	25	2	141	142	1	11.4	86.7	20.2	88.6	13.3
Osiris	13	59	43	2	117	142	25	18.1	86.7	29.9	81.9	13.3
Qian												
Ours	86	9	180	0	275	275	0	90.5	100.0	**95.0**	9.5	0.0
Mythril	23	2	187	63	275	275	0	92.0	26.7	41.4	8.0	73.3
Oyente+	63	46	143	23	275	275	0	57.8	73.3	64.6	42.2	26.7
Osiris	52	19	168	34	273	275	2	73.2	60.5	66.2	26.8	39.5

Table 6. Timestamp Dependency detection results.

Tool	TP	FP	TN	FN	Analyzed	Total	Failures	P	R	F1	FDR	FNR
SBC												
Ours	14	9	115	0	138	142	4	60.9	100.0	75.7	39.1	0.0
Mythril	12	2	126	2	142	142	0	85.7	85.7	**85.7**	14.3	14.3
Oyente+	11	2	125	3	141	142	1	84.6	78.6	81.5	15.4	21.4
Slither	14	9	112	0	135	142	7	60.9	100.0	75.7	39.1	0.0
Osiris	4	1	103	9	117	142	25	80.0	30.8	44.4	20.0	69.2
SmartCheck	1	0	128	13	142	142	0	100.0	7.1	13.3	0.0	92.9
Qian												
Ours	173	29	147	0	349	349	0	85.6	100.0	**92.3**	14.4	0.0
Mythril	59	86	90	114	349	349	0	40.7	34.1	37.1	59.3	65.9
Oyente+	1	15	160	172	348	349	1	6.2	0.6	1.1	93.8	99.4
Slither	110	117	59	63	349	349	0	48.5	63.6	55.0	51.5	36.4
Osiris	2	16	157	164	339	349	10	11.1	1.2	2.2	88.9	98.8
SmartCheck	2	1	175	171	349	349	0	66.7	1.2	2.3	33.3	98.8

Just Accepted

and refining candidate dependencies using abstract semantics is particularly important when the dependency spans multiple expressions, statements, or functions.

Table 7. Transaction Ordering Dependency (TOD) detection results.

Tool	TP	FP	TN	FN	Analyzed	Total	Failures	P	R	F1	FDR	FNR
SBC												
Ours	4	15	119	0	138	142	4	21.1	100.0	**34.8**	79.0	0.0
Mythril	1	37	101	3	142	142	0	2.6	25.0	4.8	97.4	75.0
Oyente+	2	35	102	2	141	142	1	5.4	50.0	9.8	94.6	50.0
Osiris	2	29	86	0	117	142	25	6.5	100.0	12.1	93.5	0.0
Sailfish	2	62	73	2	139	142	3	3.1	50.0	5.9	96.9	50.0
SolidiFI												
Ours	50	23	27	0	100	100	0	68.5	100.0	81.3	31.5	0.0
Mythril	21	0	50	29	100	100	0	100.0	42.0	59.2	0.0	58.0
Oyente+	36	4	46	14	100	100	0	90.0	72.0	80.0	10.0	28.0
Osiris	NA	NA	NA	NA	NA	100	NA	NA	NA	NA	NA	NA
Sailfish	40	5	43	1	89	100	11	88.9	97.6	**93.0**	11.1	2.4

Just Accepted

Transaction-ordering dependency. Table 7 reports the TOD results on SBC and SolidiFI. SBC contains only four TOD-positive contracts. Our framework detects all four and achieves the highest F1 score of 34.8%, compared with 12.1% for the strongest baseline, Osiris. Nevertheless, its precision remains 21.1% because 15 negative contracts are also reported as vulnerable. Given the small positive class, each false positive has a substantial effect on precision and F1. On SolidiFI, Sailfish records the highest F1 score of 93.0%, although it excludes 11 contracts from the analysis. Our framework analyzes all 100 contracts and detects all 50 vulnerable contracts, producing 100% recall and an F1 score of 81.3%. Among the tools that analyze the complete benchmark, this is the highest F1 score, slightly above Oyente+ at 80.0% and substantially above Mythril at 59.2%. The 23 false positives recorded by our framework show that further refinement is needed to improve TOD precision, although the semantic dependency analysis avoids the large number of missed vulnerabilities observed for Mythril and Oyente+.

Figure 15 summarizes the F1 scores across the nine evaluated dataset-vulnerability combinations. Our framework obtains the highest F1 score in seven of them. The two exceptions are timestamp dependency on SBC, where Mythril performs best, and TOD on SolidiFI, where Sailfish achieves the highest score on the subset it analyzes. Across all nine cases, our framework reports no false negatives among the analyzed contracts. At the same time, the results also show that our analysis produces more false positives in some settings, particularly for SBC arithmetic, SBC TOD, and SolidiFI TOD. Overall, the results provide a positive answer to RQ1 by showing that the semantics-based analysis achieves high vulnerability coverage, although further precision improvements are still possible in some cases.

7.1.2 ROC Curve Analysis. To gain a more intuitive understanding of the performance comparison among the tools, we plot the ROC curves for all nine tools, as shown in Figure 16. The ROC (Receiver Operating Characteristic) curve relates the True Positive Rate (TPR) to the False Positive Rate (FPR), where $\text{TPR} = \frac{\text{TP}}{\text{TP+FN}}$, $\text{FPR} = \frac{\text{FP}}{\text{TN+FP}}$, with TPR on the vertical axis and FPR on the horizontal axis. Since each tool produces a binary verdict for each

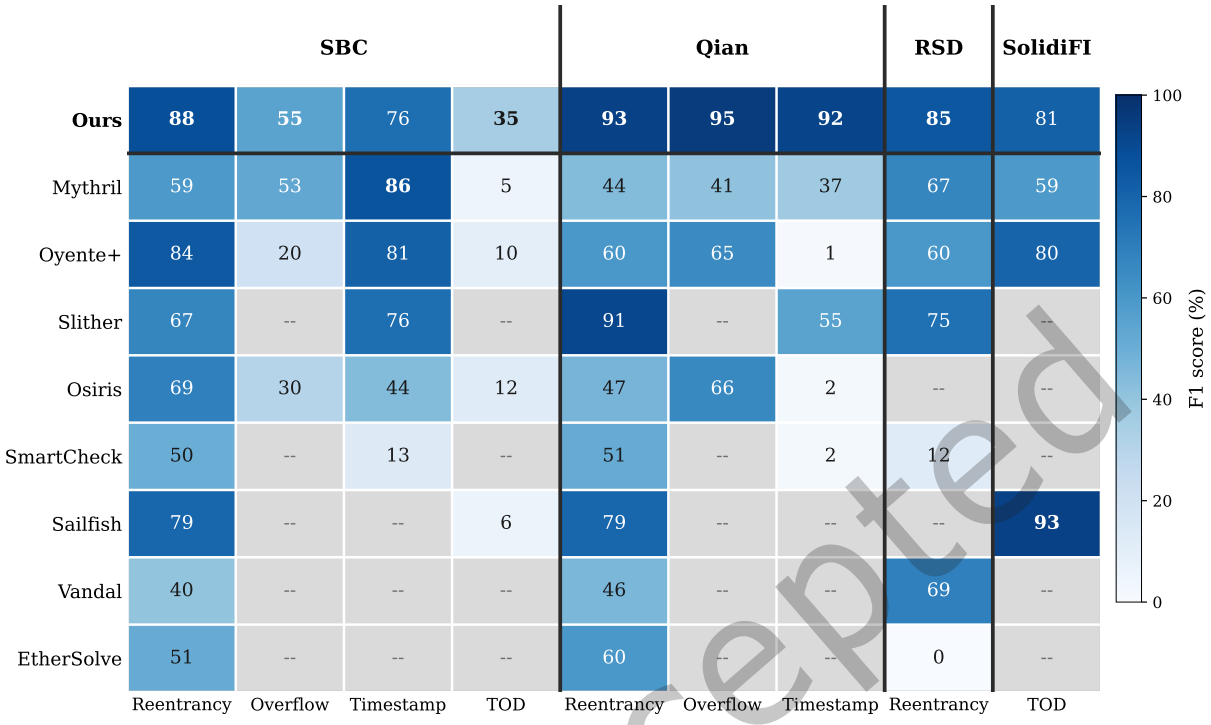


Fig. 15. Per-class F1 score (%) of each tool across the nine evaluated dataset-vulnerability combinations. Columns are grouped by dataset and vulnerability class. Darker cells indicate higher F1 scores, and the highest value in each column is shown in bold. A dash (–) marks a class for which the tool provides no detector and is therefore not evaluated.

Just Accepted

contract, every tool contributes one operating point. The area under the curve (AUC) is used as the performance indicator. The larger the value of AUC, the better the performance.

For reentrancy, our tool obtains the highest AUC on all three datasets: 0.96 on SBC, 0.96 on Qian, and 0.84 on RSD (Figure 16(a), (e), and (h)). The corresponding margins over the closest baselines are 0.03 over Oyente+ on SBC, 0.01 over Slither on Qian, and 0.13 over Slither on RSD. For integer overflow and underflow, our tool also records the highest AUC, reaching 0.90 on SBC and 0.98 on Qian (Figure 16(b) and (f)). These values exceed the closest baselines, Mythril and Osiris, by 0.07 and 0.23, respectively. For timestamp dependency, our tool obtains an AUC of 0.96 on SBC, equal to Slither, and 0.92 on Qian (Figure 16(c) and (g)). On Qian, the next highest AUC is 0.50 for SmartCheck. For TOD, our tool achieves the highest AUC on SBC, with 0.94 compared with 0.87 for Osiris (Figure 16(d)). On SolidiFI, Sailfish obtains the highest AUC of 0.94, followed by Oyente+ at 0.82 and our tool at 0.77 (Figure 16(i)). Although our tool detects all vulnerable contracts in this dataset, its larger number of false positives lowers the AUC.

These results address **RQ1** and **RQ3**. For **RQ1**, our framework achieves the highest F_1 score in seven of the nine evaluated settings and the highest or joint-highest AUC in eight settings. These results show that the semantics-based analysis generally provides a better balance between detecting vulnerable contracts and limiting false alarms than the evaluated baselines. For **RQ3**, the results across SBC, Qian, RSD, and SolidiFI show that the framework performs consistently across different datasets and vulnerability classes. The SolidiFI results also show

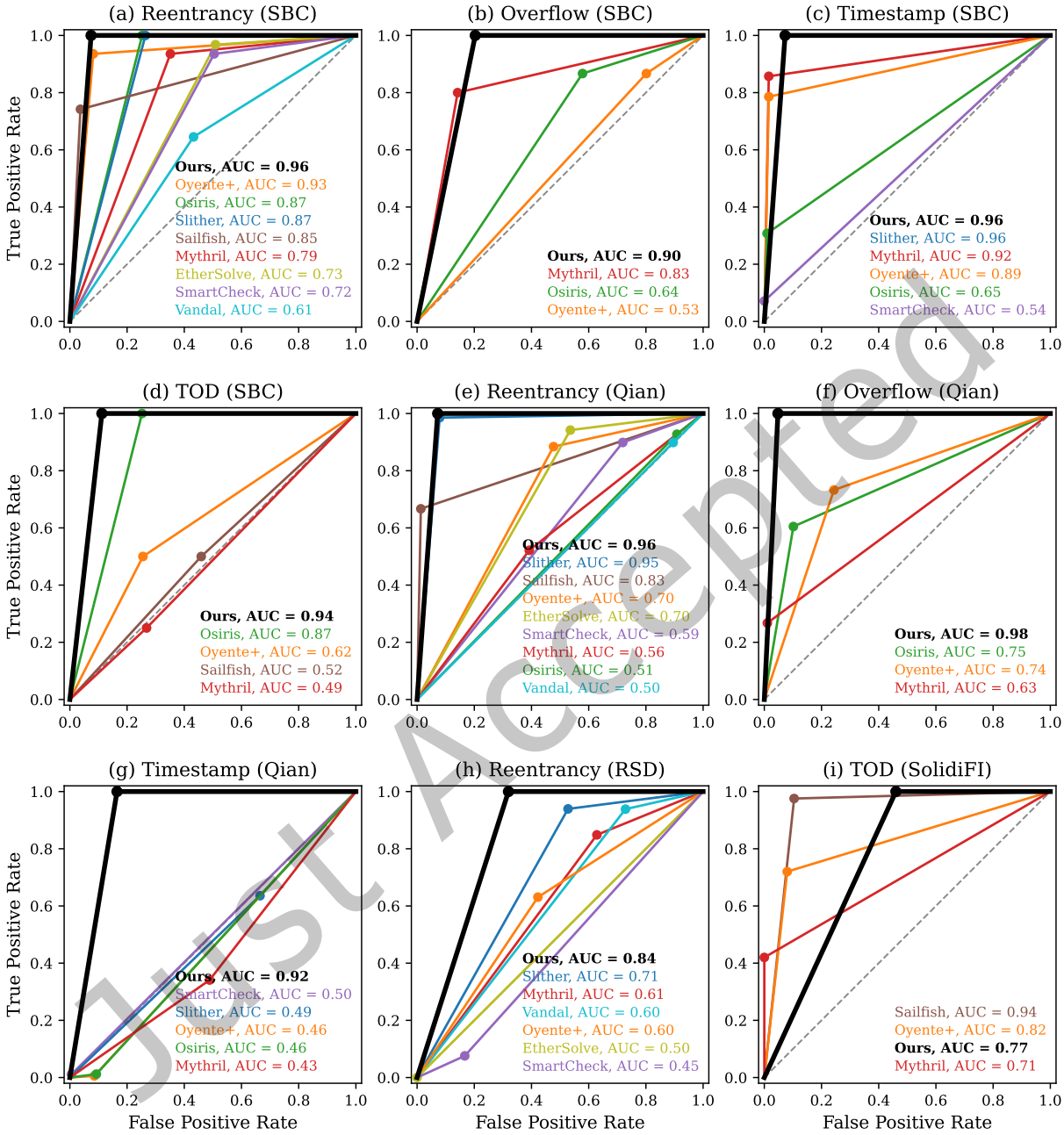


Fig. 16. ROC plots for the nine evaluated tools across the four benchmarks. Each panel represents one dataset–vulnerability setting, and each tool contributes a single operating point. Panels (a)–(d) show SBC, panels (e)–(g) show Qian, panel (h) shows RSD, and panel (i) shows SolidiFI.

Just Accepted

that this improvement is not uniform in every setting, since the complete recall achieved for TOD is accompanied by a higher number of false positives.

7.1.3 Computational Efficiency Analysis. Table 8 reports the median per-contract detection time for each tool on the four benchmarks. The baseline tools are evaluated in the same fixed, version-pinned environments used for the effectiveness evaluation. Mythril is run from its official release, while the remaining baselines are run in Docker, either from digest-pinned Docker Hub images or from local images built from pinned source revisions. For our framework, the total end-to-end analysis time is measured separately under the Interval, Octagon, and Polyhedra domains. Each analysis is repeated three times, and the mean of the three runs is used as the contract-level time. We report the median across contracts because a few contracts, particularly for the symbolic-execution tools, require much longer analysis times and would make the mean less representative of the typical runtime.

Table 8. Median per-contract detection times (seconds); "–" indicates that the tool does not support the corresponding vulnerability class or was not evaluated on that dataset.

Dataset	Mythril	Oyente+	Slither	Osiris	SmartCheck	Sailfish	Vandal	EtherSolve	Ours		
									Interval	Octagon	Polyhedra
SBC	145.68	15.98	1.00	7.34	3.62	2.25	0.37	0.40	0.99	0.99	0.99
Qian	6.15	1.27	0.64	0.30	1.48	1.12	0.37	0.36	1.10	1.17	1.17
RSD	305.00	39.76	0.65	–	2.19	–	0.52	0.37	1.61	1.75	1.68
SolidiFI	318.71	125.89	–	–	–	3.92	–	–	1.06	1.12	1.08

Across the four benchmarks, the Octagon and Polyhedra domains introduce only a small additional cost over the Interval domain. This limited overhead is mainly due to the small number of variables present in each contract. Our framework is therefore considerably faster than the symbolic-execution tools on most datasets. Mythril, for example, requires median times of 305.00 and 318.71 seconds per contract on RSD and SolidiFI, respectively, compared with approximately one to two seconds for our framework. Oyente+ also records a median time of 125.89 seconds on SolidiFI. Slither, SmartCheck, Sailfish, Vandal, and EtherSolve generally record low analysis times where their detectors are applicable, while our framework remains within a comparable range and supports all four vulnerability classes considered in this study. Although the Qian dataset contains more contracts, these contracts are generally simpler and therefore require less analysis time. In contrast, SBC contains more complex structures, including nested control flows, multiple mappings, and frequent external calls, which increase detection times. A similar pattern is observed for Osiris, whose median time increases from 0.30 seconds on Qian to 7.34 seconds on SBC.

This evaluation of computational overhead directly addresses **RQ2**. As shown in Table 8, the choice of abstract domain, namely Interval, Octagon, or Polyhedra, has only a marginal effect on the median runtime. This indicates that the framework can use relational domains without a substantial increase in analysis cost. The Interval variant also has analysis times comparable to lightweight tools such as Slither, while providing the semantic reasoning reflected in the precision and recall results reported above.

7.2 Case Studies

This section examines one representative contract for each vulnerability class. These examples illustrate how our analysis derives its results and compare them with the outputs of the baseline tools under the configurations reported in Table 9.

Reentrancy. Figure 17 shows the `LoopCrossMod` contract. In the `payAll()` function, Ether is transferred to a recipient before the corresponding entry in `balances` is cleared. A malicious recipient can therefore re-enter

Just Accepted

```
contract LoopCrossMod {
    mapping(address => uint256) public balances;
    bool private flag;
    modifier nonReentrant() {
        require(!flag, "Locked");   // checks the lock...
        _;
        flag = false;               // ...but never sets flag = true
    }
    function payAll(address[] memory recipients) nonReentrant public {
        for (uint i = 0; i < recipients.length; ++i) {
            address r = recipients[i];
            require(balances[r] > 0, "Insufficient funds");
            (bool success, ) = r.call{value: balances[r]}("");  // external call
            require(success, "Call failed");
            balances[r] = 0;        // credit cleared AFTER the call
        }
    }
    function deposit() nonReentrant public payable {
        balances[msg.sender] += msg.value;
    }
}
```

Fig. 17. The LoopCrossMod contract having Reentrancy Vulnerability.

the function during the external call while its recorded balance is still positive and receive more than one payment. The `nonReentrant` modifier does not prevent this behaviour because `flag` is never set to `true` before the function body is executed. Our analysis models the possible re-entry at the external call and applies the balance-preservation check in Algorithm 2. Since the transfer may be repeated before `balances[r]` is reset, the recipient's balance may increase without a corresponding update to the recorded credit. The balance-preservation invariant is therefore violated, and the contract is reported as reentrant. Under the evaluated configurations, Slither, Oyente+, and Vandal also report this vulnerability. Mythril, SmartCheck, and EtherSolve do not report it, whereas Osiris and Sailfish cannot analyze the contract because of compiler compatibility limitations.

Integer overflow and underflow. Figure 1 shows the `FeeAccumulator` contract. The loop repeatedly multiplies the `uint8` variable `fee` by three. For `periods >= 5`, the computed value exceeds the maximum `uint8` value of 255. Although Solidity 0.8 inserts a runtime check that reverts the transaction, the source code still contains an arithmetic condition that causes execution to fail for these inputs. Our analysis applies the interval-domain fixpoint computation of Algorithm 3. As shown in Part C of Figure 1, the abstract value of `fee` eventually widens to $[0, +\infty)$. Since this range exceeds the declared `uint8` bound, the analysis reports an overflow condition. Mythril and Oyente+ do not report an overflow for this contract, and Slither does not provide an integer-overflow detector. Osiris cannot parse the Solidity 0.8 version directly. We therefore tested the same arithmetic logic using pragma `^0.4.21`, but Osiris still produced no finding. This case illustrates the benefit of tracking source-level value ranges across loop iterations.

Timestamp dependence. Figure 2b shows the `Governmental` contract, whose analysis is presented in Figure 11 and Example 5.17. The `invest()` function stores `block.timestamp` in `lastInvestmentTimestamp`. The `resetInvestment()` function later uses this stored value to decide whether the payout can proceed. The payout decision therefore depends on a timestamp-influenced state value across two different functions. Our analysis first identifies the dependency from `block.timestamp` to `lastInvestmentTimestamp` and then to the payout condition. The abstract-state analysis confirms that the path leading to the transfer is feasible under the timestamp constraint described in Example 5.17. The contract is therefore reported as timestamp-dependent. Under the evaluated configurations, Mythril and SmartCheck do not report this case.

Just Accepted

Transaction-ordering dependence. Figure 2d shows the `EGame` contract, whose detailed analysis is presented in Figure 13 and Example 5.18. The state variable `winner` is updated in `play()` and later used as the transfer recipient in `getReward()`. Since the update in `play()` is controlled by a timestamp-based condition, the order in which the two functions are invoked can affect the recipient of the transfer. Our analysis identifies the definition–use pair over `winner`. It then verifies that the assignment in `play()` changes the relevant state, that the transfer in `getReward()` depends on `winner`, and that the update is controlled by the timestamp condition. Since all three checks in Algorithm 4 hold, the contract is reported as transaction-ordering dependent. Among the evaluated baselines, Sailfish also reports this vulnerability. Mythril and Oyente+ do not report it under the configurations used in our experiments, while Osiris cannot analyze the required Solidity version.

7.3 Reproducibility

To support reproducibility, all artifacts of this study are released in a public GitHub repository.[2] The repository contains:

- the complete source code of our analysis framework;
- the benchmark datasets and their ground-truth labels;
- the raw per-contract outputs of our tool and all baseline tools;
- the scripts used to evaluate the results and generate Tables 4–8 and Figures 15–16;
- for each Docker-based baseline, the exact registry image reference and digest for pulled images, or the Dockerfile and pinned commit for locally built images, together with the execution scripts and configuration files; and
- instructions for installation, configuration, and execution.

Our tool accepts a Solidity source file as input and produces a report indicating the presence or absence of reentrancy, integer overflow/underflow, timestamp dependency, and transaction-ordering dependency vulnerabilities. The framework automatically applies a predefined abstract-domain configuration, using the Interval domain for reentrancy and integer overflow/underflow and the Polyhedra domain for the semantic refinement of timestamp and transaction-ordering dependency candidates. Each analysis run uses a single abstract domain. A command-line option allows any vulnerability detector to be executed separately under the Interval, Octagon, or Polyhedra domain, as performed in the domain comparison for RQ2. Thus, the default analysis requires no per-contract manual domain selection or tuning. The appropriate Solidity compiler version is selected automatically from the contract's `pragma` declaration. The provided scripts analyze each benchmark contract, compare the reported verdict with its ground-truth label, and regenerate the evaluation results presented in this paper.

To ensure the reproducibility of the baseline evaluation, all tools are executed in fixed and version-pinned environments, as summarized in Table 9. Mythril v0.24.8 is run from its official release because its Docker execution repeatedly exceeds the analysis time limit. The original Oyente implementation is replaced with Oyente+ from the SmartBugs `oyente_plus` repository at commit `e225d24`. For the Docker Hub images of Slither, Osiris, SmartCheck, and Sailfish, the replication package records the exact image references and registry digests. The images for Oyente+, Vandal, and EtherSolve are built locally from their corresponding Dockerfiles and pinned source revisions: Oyente+ commit `e225d24`, Vandal commit `d2b0043`, and EtherSolve release `V1.1` (commit `59fe4412f7`). The package also provides the execution commands, timeout settings, and per-contract outputs for all baselines. For each contract, the Solidity compiler version is selected according to its `pragma` declaration.

We adopt this procedure to obtain explicit control over the tool versions, execution commands, timeout settings, and per-contract outputs, thereby avoiding ambiguity introduced by framework-level defaults. For each

[2] https://github.com/maitrikushwaha/solidity-analysis-framework-tool

Just Accepted

Table 9. Tool versions and execution environments used in the evaluation. For tools run from a registry image we report the fully qualified reference and its content digest (sha256); tools built from source list the upstream repository and the pinned commit. The execution commands are provided in the replication package.

Tool	Version	Mode	Image (with digest) or source (with commit)
Our tool	Artifact	Native	Project repository (https://github.com/maitrikushwaha/solidity-analysis-framework-tool)
Mythril	v0.24.8	Native	mythril==0.24.8 (github.com/ConsenSysDiligence/mythril official release; same tool version as image smartbugs/mythril:0.24.8)
Oyente+	e225d24	Docker	built from github.com/smartbugs/oyente_plus @ e225d24 (→ image oyente-plus-repro:latest, base python:3.11-slim)
Slither	v0.10.4	Docker	smartbugs/slither:0.10.4 @sha256:c5987e3b1de90de46ac197818dc6a17b bb61a751ba5406679c30b39ca90906d2
Osiris	d1ecc37	Docker	smartbugs/osiris:d1ecc37 @sha256:17cf6e59968191b43e330b9f4a64e70e bb1cfc34a27e7b4d16f91970ff925ec9
SmartCheck	v2.0.3	Docker	smartbugs/smartcheck:latest @sha256:3d1e348d7e8e34a9eb59cc70cd7a8020 f706c44080272e2fd92ae4ebd9b58b34
Sailfish	latest	Docker	holmessherlock/sailfish:latest @sha256:ba0770955356b0f7d9bcb25ba2c65af6 042cf0256a2fa843c1b8aa4821c5d166
Vandal	d2b0043	Docker	built from github.com/usyd-blockchain/vandal @ d2b0043 (→ image vandal:pinned-d2b0043)
EtherSolve	v1.1	Docker	built from github.com/SeUniVr/EtherSolve @ V1.1 (59fe4412f7) (→ image ethersolve-repro:v1.1)

Solidity compiler versions 0.4.11–0.8.28 are selected according to each contract's pragma declaration. Images under the smartbugs/* and holmessherlock/* namespaces are pulled from Docker Hub at the listed digest; Oyente+, Vandal, and EtherSolve are built from the cited upstream source at the pinned commit. Mythril is executed from its official v0.24.8 release (same tool version as image smartbugs/mythril:0.24.8).

Docker-based baseline, the replication package provides either the registry image reference and digest or the Dockerfile and pinned source revision, together with the execution command, timeout setting, and raw outputs.

7.4 Limitations

In this section, we mention some limitations of the current proposal, which can easily be extended in future work. These limitations mainly concern the developed prototype.

(1) Even though Table 1 shows the abstract syntax and concrete semantics for a subset of Solidity constructs, the approach is not limited to this subset only. The same can be extended to other Solidity constructs such as arrays, structures, and function modifiers, whose formal semantics are already well established in the literature [2, 50].
(2) Even though our prototype covers most of the language constructs used in practice, at this moment the preliminary version does not handle nested mappings, inheritance with virtual dispatch, custom errors, and library calls. These can be extended in future versions of the tool.

Just Accepted

(3) In reentrancy detection, the numerical fixpoint (Algorithm 2) checks the balance-preservation invariant over the recognized Ether-transfer encoding and does not track boolean lock variables. As a result, a contract containing an Ether-bearing external call that is protected by a correctly implemented mutex may still be reported as reentrant, because the fixpoint cannot determine that the lock prevents a second entry. This is one source of the false positives we observe on the RSD benchmark. The current implementation also uses a structural check to identify such locks; however, this check does not cover all mutex patterns. Since our abstract interpretation is based on numerical domains, extending the fixpoint to a combined Boolean and numerical domain is left as future work.
(4) As in any sound static analysis, false positives remain a practical issue because the analysis safely over-approximates contract behavior. In our setting, such false positives may be reduced using standard precision-improving strategies from Abstract Interpretation, such as input partitioning or reduced products of abstract domains, at the cost of additional computational effort. We leave this trade-off between precision and computational cost as part of future work.

8 Related Work

Early research on smart contract vulnerability detection relied mainly on symbolic execution. Oyente [29] was among the first tools to construct control flow graphs from EVM bytecode and apply symbolic execution to detect security issues. It targets reentrancy, integer-related bugs, timestamp dependency, and transaction-ordering dependency, but its bounded path exploration may leave some execution paths unexplored. Oyente+ extends the original implementation with improved support for recent compiler and EVM versions while retaining its symbolic-execution approach. Osiris [46] builds on symbolic execution and taint analysis, with its original design centred on integer bugs such as overflow, underflow, truncation, and signedness errors. Mythril [33] performs symbolic execution over EVM bytecode and supports a broader set of vulnerability classes, although its results remain dependent on the configured exploration bounds. Slither [18] analyzes Solidity source code using SSA-based data-flow analysis and predefined detectors. SmartCheck [44] applies XPath-based rules to an intermediate representation of the source code, whereas Sailfish [5] combines a storage-dependency graph with symbolic value-summary reasoning for reentrancy and transaction-ordering bugs. Vandal [6] decompiles EVM bytecode into a logic-based intermediate representation and uses Datalog specifications to express security analyses. EtherSolve [38] constructs a control-flow graph from EVM bytecode through symbolic stack execution and applies a CFG-based analysis to detect reentrancy vulnerabilities.

Beyond the tools used in our experimental comparison, several other approaches have been proposed. Maian [34] uses inter-procedural analysis to trace contract-level properties, such as whether a contract can be destructed or whether Ether can be locked. Securify [47] combines symbolic reasoning with abstract interpretation over EVM bytecode to infer semantic facts about contract behaviour. ZEUS [25] translates Solidity source to LLVM bitcode for formal verification, but this translation does not cover all Solidity constructs and EVM-specific features. Pluto [30] builds inter-contract control-flow graphs and applies rule-based checks. SoliDetector [23] couples abstract interpretation with a knowledge graph to capture logical relationships, though each new vulnerability type requires a custom validation mechanism. Several survey papers [1, 28, 42, 45, 48] provide broader overviews of this area.

Across these approaches, a general trade-off can be observed. Pattern-based tools such as Slither and SmartCheck check whether the code matches a known vulnerability template; they can therefore raise a false alarm when the code looks suspicious but is actually safe, or miss a vulnerability that does not fit the expected pattern. Symbolic-execution tools such as Mythril and Oyente+ explore execution paths under configured bounds and resource limits, so vulnerabilities hidden in deep or complex call sequences may go undetected; Osiris shares similar bounded-exploration limitations within its original integer-arithmetic scope. Our approach instead uses

Just Accepted

abstract interpretation to compute a sound over-approximation of reachable program states within the modeled Solidity fragment, rather than exploring individual paths, and can therefore, in principle, reason about behaviours that bounded path exploration would miss.

Methodologically, the closest line of work is the application of abstract interpretation to smart contracts written in general-purpose languages, as in the analyses of Go-based Hyperledger Fabric contracts that address determinism [37], read-write conflicts [35], and untrusted cross-contract invocations [36]. These works share our foundational technique but differ in the target language (Solidity), the execution model (EVM and Ethereum-specific constructs such as msg.sender, block.timestamp, and external calls with Ether transfer), and the vulnerability classes addressed. Finally, semi-automated formal-verification approaches based on interactive theorem provers such as F*, K [41], and Isabelle/HOL offer strong guarantees but require substantial manual effort and are not directly comparable to fully automated tools.

9 Conclusions

In this paper, we present a semantics-driven static analysis framework for detecting smart contract vulnerabilities by extending the theory of Abstract Interpretation to the Solidity language. The proposed method enables sound reasoning about contract behaviour under adversarial blockchain conditions by formally modelling both concrete and abstract semantics across multiple numerical domains. The framework captures semantic dependencies that symbolic or rule-based tools often miss by integrating control- and data-flow analyses within an augmented CFG. Experimental results obtained using the APRON library across four benchmark datasets (SBC, Qian, RSD, and SolidiFI) demonstrate that the tool effectively detects reentrancy, integer overflow and underflow, timestamp dependency, and transaction ordering dependency vulnerabilities. Compared with the strongest baseline in each dataset-vulnerability combination, our tool achieves the highest F1 score in seven of the nine combinations. The average F1 score gains are approximately 5.3, 15.0, 13.6, and 5.5 percentage points for reentrancy, integer overflow and underflow, timestamp dependency, and transaction-ordering dependency, respectively. The maximum AUC gain is approximately 0.41. The two exceptions are timestamp dependency on SBC and transaction-ordering dependency on SolidiFI, where a baseline achieves a higher F1 score. While the current method focuses on four primary vulnerability classes, extending the approach to additional categories and integrating it with dynamic analysis constitute promising directions for future work.

Acknowledgement

This work is supported by the Visvesvaraya Phase II Ph.D. Scheme, Ministry of Electronics and Information Technology (MeitY), Government of India, being implemented by Digital India Corporation. The authors also thank the ANRF Core Research Grant (Grant Number: CRG/2022/005794) by the Government of India for carrying out this research. We thank Arnab Mukherjee, Kartik Kaushik and Aishwarya Pandey for their support during tool development.

References

[1] Mouhamad Almakhour, Layth Sliman, Abed Ellatif Samhat, and Abdelhamid Mellouk. 2020. Verification of smart contracts: A survey. *Pervasive and Mobile Computing* 67 (2020), 101227.

[2] Jim Alves-Foss and Fong Shing Lam. 2002. Dynamic denotational semantics of Java. In *Formal syntax and semantics of Java*. Springer, 201–240.

[3] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. 2017. A survey of attacks on ethereum smart contracts (sok). In *International conference on principles of security and trust*. Springer, 164–186.

[4] Mueller Bernhard. 2018. A Disastrous Vulnerability Found in Smart Contracts of BeautyChain (BEC). *Retrieved June* 20 (2018), 2022.

[5] Priyanka Bose, Dipanjan Das, Yanju Chen, Yu Feng, Christopher Kruegel, and Giovanni Vigna. 2022. Sailfish: Vetting smart contract state-inconsistency bugs in seconds. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 161–178.

Just Accepted

[6] Lexi Brent, Anton Jurisevic, Michael Kong, Eric Liu, Francois Gauthier, Vincent Gramoli, Ralph Holz, and Bernhard Scholz. 2018. Vandal: A scalable security analysis framework for smart contracts. *arXiv preprint arXiv:1809.03981* (2018).
[7] Huashan Chen, Marcus Pendleton, Laurent Njilla, and Shouhuai Xu. 2020. A survey on ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Computing Surveys (CSUR)* 53, 3 (2020), 1–43.
[8] Yizhou Chen, Zeyu Sun, Zhihao Gong, and Dan Hao. 2024. Improving smart contract security with contrastive learning-based vulnerability detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–11.
[9] NV Chernikoba. 1968. Algorithm for discovering the set of all the solutions of a linear programming problem. *U. S. S. R. Comput. Math. and Math. Phys.* 8, 6 (1968), 282–293.
[10] Agostino Cortesi and Matteo Zanioli. 2011. Widening and narrowing operators for abstract interpretation. *Computer Languages, Systems & Structures* 37, 1 (2011), 24–42.
[11] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. of the 4th symposium on Principles of programming languages*. 238–252.
[12] Patrick Cousot and Radhia Cousot. 1979. Systematic design of program analysis frameworks. In *Proceedings of the 6th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. ACM Press, San Antonio, Texas, 269–282.
[13] Patrick Cousot and Nicolas Halbwachs. 1978. Automatic Discovery of Linear Restraints Among Variables of a Program. In *Proceedings of the POPL '78*. 84–96.
[14] Phil Daian. 2016. Analysis of the DAO exploit. *Hacking, Distributed* 6 (2016).
[15] Dipanjan Das, Priyanka Bose, Nicola Ruaro, Christopher Kruegel, and Giovanni Vigna. 2022. Understanding security Issues in the NFT Ecosystem. In *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security*. 667–681.
[16] Thomas Durieux et al. 2020. Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International conference on software engineering*. 530–541.
[17] Gernot Salzer et al. [n. d.]. SB Curated dataset. https://github.com/smartbugs/smartbugs-curated. [Accessed 24-07-2024].
[18] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 8–15.
[19] Cuifeng Gao, Ao Chen, Chengze Wu, Wenzhang Yang, Jiaming Ye, and Yinxing Xue. 2025. Are Static Analysis Tools Still Working during the Evolution of Smart Contracts? A Comprehensive Empirical Study. *ACM Transactions on Software Engineering and Methodology* (2025).
[20] Asem Ghaleb and Karthik Pattabiraman. 2020. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 415–427.
[21] Raju Halder and Agostino Cortesi. 2013. Abstract program slicing of database query languages. In *Proceedings of the 28th Annual ACM Symposium on Applied Computing (SAC '13)*. ACM, Coimbra, Portugal, 838–845.
[22] Raju Halder and Agostino Cortesi. 2013. Abstract program slicing on dependence condition graphs. *Science of Computer programming* 78, 9 (2013), 1240–1263.
[23] Tianyuan Hu, Bixin Li, Zhenyu Pan, and Chen Qian. 2023. Detect defects of solidity smart contract based on the knowledge graph. *IEEE Transactions on Reliability* 73, 1 (2023), 186–202.
[24] Bertrand Jeannet and Antoine Miné. 2009. Apron: A library of numerical abstract domains for static analysis. In *International Conference on Computer Aided Verification*. Springer, 661–667.
[25] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. 2018. Zeus: analyzing safety of smart contracts. In *Ndss*. 1–12.
[26] Alexander Küchler, Leon Wenning, and Florian Wendland. 2023. AbsIntIO: Towards Showing the Absence of Integer Overflows in Binaries using Abstract Interpretation. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security (ASIA CCS '23)*. Association for Computing Machinery, Melbourne, VIC, Australia, 247–258.
[27] Maitri Kushwaha, Arnab Mukherjee, Aishwarya Pandey, and Raju Halder. 2024. Semantics-Based Static Vulnerability Detection in Solidity Using Abstract Interpretation. In *International Conference on Information Systems Security*. Springer, 265–285.
[28] Detian Liu, Jianbiao Zhang, Yifan Wang, Hong Shen, Zhaoqian Zhang, and Tao Ye. 2025. Blockchain Smart Contract Security: Threats and Mitigation Strategies in a Lifecycle Perspective. *Comput. Surveys* (2025).
[29] Loi Luu et al. 2016. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 254–269.
[30] Fuchen Ma, Zhenyang Xu, Meng Ren, Zijing Yin, Yuanliang Chen, Lei Qiao, Bin Gu, Huizhong Li, Yu Jiang, and Jiaguang Sun. 2021. Pluto: Exposing vulnerabilities in inter-contract scenarios. *IEEE Transactions on Software Engineering* 48, 11 (2021), 4380–4396.
[31] Isabella Mastroeni and Damiano Zanardini. 2008. Data dependencies and program slicing: from syntax to abstract semantics. In *Proceedings of the 2008 ACM SIGPLAN symposium on Partial evaluation and semantics-based program manipulation*. 125–134.
[32] Antoine Minè. 2006. http://www.astree.ens.fr/. The Octagon Abstract Domain. *Higher Order Symbol. Comput.* 19, 1 (2006. http://www.astree.ens.fr/), 31–100.
[33] Bernhard Mueller. 2018. Smashing ethereum smart contracts for fun and actual profit. In *Proceedings of the HITB Security Conference*.

Just Accepted

[34] Ivica Nikolić, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. 2018. Finding the greedy, prodigal, and suicidal contracts at scale. In *Proceedings of the 34th annual computer security applications conference*. 653–663.
[35] Luca Olivieri, Luca Negrini, Vincenzo Arceri, Pietro Ferrara, and Agostino Cortesi. 2025. Detection of Read-Write Issues in Hyperledger Fabric Smart Contracts. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*. 329–337.
[36] Luca Olivieri, Luca Negrini, Vincenzo Arceri, Pietro Ferrara, Agostino Cortesi, and Fausto Spoto. 2025. Static Detection of Untrusted Cross-Contract Invocations in Go Smart Contracts. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*. 338–347.
[37] Luca Olivieri, Fabio Tagliaferro, Vincenzo Arceri, Marco Ruaro, Luca Negrini, Agostino Cortesi, Pietro Ferrara, Fausto Spoto, and Enrico Talin. 2022. Ensuring determinism in blockchain software with GoLiSA: an industrial experience report. In *Proceedings of the 11th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. 23–29.
[38] Michele Pasqua, Andrea Benini, Filippo Contro, Marco Crosara, Mila Dalla Preda, and Mariano Ceccato. 2023. Enhancing Ethereum smart-contracts static analysis by computing a precise Control-Flow Graph of Ethereum bytecode. *Journal of Systems and Software* 200 (2023), 111653.
[39] Peng Qian, Zhenguang Liu, Yifang Yin, and Qinming He. 2023. Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode. In *Proceedings of the ACM web conference 2023*. 2220–2229.
[40] Dalila Ressi, Alvise Spanò, Matteo Rizzo, Lorenzo Benetollo, and Sabina Rossi. 2026. Reentrancy Detection in the Age of LLMs. *arXiv preprint arXiv:2603.26497* (2026).
[41] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An overview of the K semantic framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434.
[42] Sarwar Sayeed, Hector Marco-Gisbert, and Tom Caira. 2020. Smart contract: Attacks and protections. *Ieee Access* 8 (2020), 24416–24427.
[43] Matt Suiche. 2017. The $280 M Ethereum's parity bug. *A critical security vulnerability in Parity multi-sig wallet* (2017).
[44] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. 2018. Smartcheck: Static analysis of ethereum smart contracts. In *Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain*. 9–16.
[45] Palina Tolmach, Yi Li, Shang-Wei Lin, Yang Liu, and Zengxiang Li. 2021. A survey of smart contract formal specification and verification. *ACM Computing Surveys (CSUR)* 54, 7 (2021), 1–38.
[46] Christof Ferreira Torres, Julian Schütte, and Radu State. 2018. Osiris: Hunting for integer bugs in ethereum smart contracts. In *Proceedings of the 34th annual computer security applications conference*. 664–676.
[47] Petar Tsankov, Andrei Dan, Dana Drachsler-Cohen, Arthur Gervais, Florian Buenzli, and Martin Vechev. 2018. Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 67–82.
[48] Fernando Richter Vidal, Naghmeh Ivaki, and Nuno Laranjeiro. 2024. Vulnerability detection techniques for smart contracts: A systematic literature review. *Journal of Systems and Software* 217 (2024), 112160.
[49] Zexu Wang, Jiachi Chen, Yanlin Wang, Yu Zhang, Weizhe Zhang, and Zibin Zheng. 2024. Efficiently detecting reentrancy vulnerabilities in complex smart contracts. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 161–181.
[50] Glynn Winskel. 1993. *The formal semantics of programming languages: an introduction*. MIT press.
[51] Zibin Zheng, Jianzhong Su, Jiachi Chen, David Lo, Zhijie Zhong, and Mingxi Ye. 2024. Dappscan: building large-scale datasets for smart contract weaknesses in dapp projects. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1360–1373.

Received 8 September 2026; revised 7 July 2026; accepted 14 December 2025

Just Accepted